

GUI - lecture notes

with example programs from archive *java_examples*

Tomasz R. Werner

Warsaw, April 3, 2025 1:03

Contents

	Page
1 Regular expressions	1
1.1 Basic concepts	1
1.1.1 Classes	1
1.1.2 Predefined classes	2
1.1.3 Special locations	2
1.1.4 Quantifiers	2
1.2 Regular expressions in methods of class <i>String</i>	3
1.3 Patterns and Matchers	5
1.4 Capturing groups	6
1.5 Option flags	9
1.5.1 <i>MULTILINE</i>	9
1.5.2 <i>DOTALL</i>	10
1.5.3 <i>CASE_INSENSITIVE</i> and <i>UNICODE_CASE</i>	10
1.5.4 <i>UNICODE_CHARACTER_CLASS</i>	11
1.6 Some examples	11
2 Basic data structures	20
2.1 Singly linked lists	20
2.2 Stacks	23
2.3 Queues	25
3 Abstract classes, interfaces, inner classes and lambdas	28
3.1 Abstract classes	28
3.2 Interfaces	31
3.3 Inner and anonymous classes	46
3.3.1 Non-static inner classes	46
3.3.2 Static inner classes	47
3.3.3 Anonymous classes	49
3.4 Lambdas	52
3.5 More examples	54
4 Introduction to generic classes	58
4.1 Type parameters	58
4.2 Bound types	67
5 <i>Enum</i> types	70
5.1 Basic definitions	70
5.2 Fields, constructors and methods in enumerations	74
5.3 Enumerations implementing an interface	77
6 Introduction to collections	79
6.1 Collections	79
6.2 Maps	80
6.3 Iterators	81
6.4 Examples	84
6.5 Importance of <i>equal</i> and <i>hashCode</i> methods	88
7 Streams	92
7.1 Introduction	92
7.2 Creating streams	93
7.3 Intermediate operations	94
7.4 Terminal operations	95
7.5 Examples	97

8	Method references	109
9	Functional interfaces	112
9.1	Consumers	112
9.2	Functions	113
9.3	Operators	113
9.4	Predicates	114
9.5	Suppliers	114
9.6	Example	115
10	List of listings	116
	Index	118

Regular expressions

Regular expression (**regex**) is a sequence of characters which defines a pattern that we want to search for in a string (generally, in a text which may be arbitrary long). Regexes are ‘compiled’ into a form resembling functions and executed by the so called *regular expression engines* — almost all contemporary languages support regular expressions (built into the language or as part of their standard libraries). The theory behind regexes is rather involved and its full understanding requires quite advanced mathematical knowledge; it was developed by an outstanding logician Stephen Cole Kleene (pronounced KLAY-nee) in 1950s and first used in practice in early implementations of Unix text processors and utility programs (Ken Thompson). Almost all modern implementations of regular expression engines are based on Larry Wall’s implementation in his Perl programming language (late 1980s).

Details on the Java implementation: see Oracle’s documentation.¹

1.1 Basic concepts

Suppose we are looking for a word, say ‘elephant’, in a text. Then the regular expression which defines this word will be just `"elephant"`. But what if we have, in our text, the word ‘Elephant’? Of course, this won’t match, because the first letter differs. Or, we look for ‘cat’ but only if it is a separate word, not part of another word (like in ‘tomcat’ or ‘caterpillar’). Or, we are looking for numbers (sequences of digits) but we don’t know in advance what numbers occur in our text and of what length (number of digits) they are. All these problems can be easily solved with the help of regular expressions which allow us to formulate such requirements as ‘a sequence of letters’, ‘any uppercase letter followed by a dot’, ‘a sequence of at least four but at most seven digits the first of which is not 0’, ‘two words separated by one or more spaces or TAB characters’, etc.

1.1.1 Classes

Classes define sets of characters. They are specified in square brackets — a hyphen between characters denotes a range, a ‘hat’ (^) at the beginning denotes negation, the `&&` symbol stands for ANDing. For example:

- `[abc]` — set of three letters: ‘a’, ‘b’ and ‘c’,
- `[a-d]` — set of four letters: ‘a’, ‘b’, ‘c’ and ‘d’,
- `[a-cu-z]` — set of lowercase letter from ranges `[a-c]` and `[u-z]`,
- `[a-zA-Z]` — set of lower- and uppercase Latin letter,
- `[a-zA-Z0-9_]` — set of all Latin letter and digits, and underscore,
- `[^0-9]` — any character, but *not* a digit,
- `[a-z&&[^i-n]]` — any character in the range `[a-z]` but simultaneously *not* in the range `[i-n]` (therefore equivalent to `[a-ho-z]`).

As we can see, there is a way to AND, but what about ORing? This can also be achieved with symbol `|`: regex `cat|dog` will look for ‘cat’ OR ‘dog’.

¹<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/regex/Pattern.html>

1.1.2 Predefined classes

Some most useful classes are predefined and denoted by special symbols; sometimes it is just one letter after a backslash — in these cases uppercase letter means the same as the corresponding lowercase symbol, but negated. For example:

- `\d` — any digit, `\D` — any non-digit,
- `\s` — any white character (space, tab, new line), `\S` — anything except white character,
- `\w` — any Latin letter, digit or underscore, `\W` — negation of `\w` (also non-Latin letters if `UNICODE_CHARACTER_CLASS` option is enabled, see sec. 1.5.4),
- `.` — any character except the new line (or including it, if `DOTALL` option has been selected, see sec. 1.5.2),
- `\p{P}` — any punctuation character, `\P{P}` — anything but not punctuation,
- `\p{L}` — any letter, in any language, `\P{L}` — not a letter,
- `\p{Ll}` — any lowercase letter, in any language,
- `\p{Lu}` — any uppercase letter, in any language,
- etc.,

NOTE: As we have seen, and will see it again quite often, backslash symbol (`\`) is very common in regular expression. It must be seen by the regex engine, but there is a problem: inside string literals it has a special meaning for the compiler (or rather its part – *tokenizer*.) How to enter it into our string? Just double it: the symbol `\\` inside a string literal denotes a *single* backslash!

1.1.3 Special locations

There are symbols which denote not characters but rather special locations in the text being analyzed. For example:

- `\b` — word boundary — just before or just after a word (a word is a sequence of characters matching `\w`),
- `^` — beginning of the text (or of a line, if `MULTILINE` option is enabled, see sec. 1.5.1),
- `$` — end of the text (or of a line, if `MULTILINE` option is enabled).

1.1.4 Quantifiers

You can put a so called *quantifier* just after an element of a regex. It then determines a possible number of repetitions of this element. For example:

- `+` — once or more,
- `*` — any number of occurrences (including zero),
- `?` — once or not at all,
- `{n,m}` — number of occurrences in the range $[n, m]$,
- `{n}` — exactly `n` occurrences.
- `{n,}` — at least `n` occurrences.

All these quantifiers are by default **greedy**. It means that the regex engine will try to find the *longest* possible match. For example, if our regex is `a.*z` and the text is `"abzcزدz"`, then the whole text will be found as the match, even though substrings `"abz"` and `"abzcz"` would be also possible (but are shorter). If this is not what we

want, we can make a quantifier **reluctant**, i.e., it will try to find the *shortest* match — in the above example `"abz"` would be found. To make a quantifier reluctant, we just add a question mark (?). Thus, continuing the above example, the regex `a.*?z` would find the shortest match (`"abz"`).

There exist the third kind of quantifiers, the so called **possessive** quantifiers, denoted by a plus symbol (+). It is, in a sense, even more greedy than greedy quantifier, because it never steps back. Normally, for a greedy quantifier, something like `.*` consumes everything and then, if there is no match, the matcher slowly backs off: it makes one step backward to see if there is a match now, if not, it steps back one more character again, and so on.

For example, let us assume that the regex is `"a.*bc"` (so `.*` is, by default, greedy) and the text is `"abcdbc"`. First, after 'a', the `.*` will consume everything. But there is no 'bc' after 'everything', so the matcher steps back one character and now there is 'c' at the end. This is still not 'bc', so the matcher makes one more step back and now it has 'bc' at the end — matching succeeds with the whole text `"abcdbc"` as the match.

Now suppose the quantifier is reluctant: `"a.*?bc"`. After 'a' the matcher consumes the shortest substring matching `.*`, that is nothing. There is no match, because there is no 'bc', so the matcher makes one step *forward* and indeed there is 'b'. After making one more step, there is 'bc', so matching succeeds and the matched substring is `"abc"`.

Now the possessive case: `"a.*+bc"`. The `.*` consumes everything, and there is no 'bc' after that. Possessive matcher never steps back, so the matching fails. Generally, you can live without possessive quantifiers, and in fact in many languages they are not supported at all. However, when they are appropriate, they make the whole process of searching for a match faster (because the matcher does not have to remember intermediate states, as it will never need to go back).

1.2 Regular expressions in methods of class *String*

There are some very useful methods in class **String** which use regular expressions:

1. method **String::split**;
2. method **String::replaceAll**;
3. method **String::matches**;

The first, **split**, invoked on a string, takes a regex and then splits the string into parts separated by substrings matching the regex — it returns an array of **Strings**. Note that the regex specifies *separators*, not what we are looking for! For example,

```
"Łódź - 0.7; London - 8.8; Tokyo - 13.6".split("\\P{L}+")
```

will produce a three-element array containing the names of the three cities: the separator here is 'non-empty sequence of any non-letters' (note the capital 'P'). Note that there is a separator (sequence of non-letters) at the end of the input string. Therefore, there should be an empty string as the last element of the resulting array of strings. However, trailing empty strings are discarded. This does not apply to leading separator: in such a case, we *will* get an empty string as the first element of the array.

Regexes can also be ORed, as the following example illustrates: here, the separator is defined as non-empty sequence of white characters (between word boundaries) OR a punctuation mark surrounded, perhaps, by sequences of white characters:

Listing 1

GXP-Split/Splitting.java

```

1 import java.io.BufferedReader;
2 import java.io.IOException;
3 import java.nio.file.Files;
4 import java.nio.file.Paths;
5 import static java.nio.charset.StandardCharsets.UTF_8;
6
7 public class Splitting {
8     public static void main(String[] args) {
9         try (
10             BufferedReader br = Files.newBufferedReader(
11                 Paths.get("Splitting.txt"), UTF_8)
12         ) {
13             String line;
14             while ((line = br.readLine()) != null) {
15                 String[] array = line.split(
16                     "(\\b\\s+\\b|\\b\\s*\\p{P}\\s*\\b)");
17                 System.out.print("|");
18                 for (String s:array) System.out.print(s+"|");
19                 System.out.println();
20             }
21         } catch (IOException e) {
22             System.out.println(e.getMessage());
23             System.exit(-1);
24         }
25     }
26 }

```

If the file contains

```

John,Mary    Charles    ;    Zoe
carrot  parsley:potato

```

then the program prints

```

| John|Mary|Charles|Zoe|
| carrot|parsley|potato|

```

The second method, **replaceAll**, takes a regex and a string and returns a string with all occurrences of substrings that match the regex replaced by the given string, e.g.,

```
"cat, caterpillar, tomcat, cat".replaceAll("\\bcat\\b", "dog")
```

will return "dog, caterpillar, tomcat, dog" (note that cat in tomcat will not be replaced because there is no word boundary before the letter 'c', and similarly for caterpillar). Invoking `s.replaceAll(regex, rep)` on a string `s` is equivalent to

```
Pattern.compile(regex).matcher(s).replaceAll(rep).
```

In particular, we can refer to capture groups, specified in the regex string, in the replacement string (see sec. 1.4 on page 6).

The third method, **matches**, takes a regex and answers the question 'does the whole string match the regex'. For example

```
"Madagascar!".matches("\\p{L}+")
```


will return **false**, because the string contains a character which is not a letter, while
`"Madagascar!".matches("\\p{L}+.*)"`
 will return **true**.

1.3 Patterns and Matchers

Regexes, before being used, must be ‘compiled’ (in the case of the aforementioned methods of class **String**, this is done automatically). This is done by invoking the static method of class **Pattern compile(regex)**: it returns an object of type **Pattern** which represents the ‘compiled’ form of our regex (we can view it as some sort of a function). Having a compiled regex (pattern), we invoke on it **matcher(text)** where `text` is the text to be analyzed (as a string). It returns an object of type **Matcher** representing the result:

```
String text = "A text";
String reg  = "a regex";
Pattern pat = Pattern.compile(reg);
Matcher m = pat.matcher(text);
```

or, if we do not need to reuse the pattern:

```
String text = "A text";
String reg  = "a regex";
Matcher m = Pattern.compile(reg).matcher(text);
```

Very often, we want to use the same pattern many times, but for different texts — for example for each line of a text file. We then compile our regex once, and then we can call **matcher** on the same object of type **Pattern** and get matchers corresponding to subsequent lines. Or, we can even create a matcher once only, passing, for example, an empty string as a text, and then reset it for subsequent lines:

```
Matcher matcher = null;
try {
    matcher = Pattern.compile("a regex").matcher("");
} catch (PatternSyntaxException e) {
    System.err.println("Wrong pattern?");
    e.printStackTrace();
    System.exit(1);
}

try (
    BufferedReader br =
        Files.newBufferedReader(
            Paths.get("file.txt"), UTF_8)
) {
    String line;
    while ((line = br.readLine()) != null) {
        matcher.reset(line);
        // ... processing information
        // ... from a single line
    }
} catch (IOException e) {
    // ...
}
```

By invoking `matcher`'s various methods, we can extract the information we need. Two basic methods which actually do the search are **matches** and **find**:

- **matches()** checks if the whole text matches the given regex (the one, on which we have called **matcher**) and returns **true** or **false**; note that it is not enough that the text contains some substrings matching the pattern — it must be the whole text;
- **find()** looks for *substrings* matching the regex (see examples below) and also returns **true** or **false**. It can be called several times; each time we invoke it, it starts searching from the location within the string after the previous match.

Matcher objects always remember the location where the last operation has finished. For successful invocation of **matches**, it will probably be the end of the input text, for an unsuccessful — the place where the matcher ‘realized’ that **matches** cannot succeed. Also for **find**, the location of the last successful match is remembered, so the next invocation of **find** will start looking for the next match. If we want to start from the beginning (for example, after calling **matches**), we can invoke `matcher.reset()`. The **reset** function is overloaded — a version taking a text makes the matcher ‘forget’ the old text and sets a new one that will be analyzed.

1.4 Capturing groups

Capturing group allows us to remember a part of text matching a pattern, so that we can use it later. A group is created when a part of a regex is enclosed in a pair of round parentheses. For example, the following regex

```
"(\\p{L}+)\\P{L}+(\\p{L}+)"
```

will match a string containing two words separated by a non-empty sequence of non-letters. However, both words will be remembered by the matcher as group number 1 and group number 2, and after a successful match we can get them by calling `matcher.get(1)` and `matcher.get(2)`, respectively. Groups may be nested and are numbered starting from 1, group number 0 being the whole substring matched, containing all the groups defined inside. Numbers are assigned according to the order in which opening parentheses of the groups are encountered — each group extends to the corresponding closing parenthesis. For example, in the program below, there will be three groups:

- one containing the first word, after, perhaps, some leading spaces, and followed by a sequence of any characters (which will not enter any group);
- then a group with two sequences of digits separated by a hyphen;
- then a group consisting of only the second of these two sequences of digits.

```
String reg = "\\s*(\\w+).*(\\d+-(\\d+))";
//           1       2       3
String text = " Einstein Albert, 1879-1955";
Matcher m = Pattern.compile(reg).matcher(text);
System.out.println("Matches? " + m.matches());
System.out.println("# of groups " + m.groupCount());
for (int i = 1; i <= m.groupCount(); ++i)
    System.out.println(i + ": " + m.group(i));
```

The program prints

```
Matches? true
# of groups 3
1: Einstein
2: 1879-1955
3: 1955
```

Note the use of **groupCount** method — it reports the number of matched groups, *not* counting the special **group(0)**. Note also that in this regex we used the reluctant **.*?** — without the question mark, **.*** would consume the first three digits of the first number!

Instead of numbering groups, we can assign names to them; the syntax is **(?<name>**, where **name** is any unique name. We then refer to such groups by invoking **matcher.group("name")**. Both ways of accessing groups, by their number or by names, are illustrated in the program below:

Listing 2

GXX-RegGroups/RegGroups.java

```
1 import java.util.regex.Matcher;
2 import java.util.regex.Pattern;
3
4 public class RegGroups {
5     public static void main (String[] args) {
6         String str = "12-07-2014 xx 6-6-2010 yy 1-11-2011";
7         String pat1 = "(\\d{1,2})-" +
8                       "(\\d{1,2})-" +
9                       "(\\d{4})";
10        String pat2 = "(?<day>\\d{1,2})-" +
11                      "(?<month>\\d{1,2})-" +
12                      "(?<year>\\d{4})";
13
14        Matcher m = Pattern.compile(pat1).matcher(str);
15        System.out.println("Unnamed groups");
16        while (m.find()) {
17            System.out.println(m.groupCount()+" groups:");
18            System.out.print("D: "+m.group(1)+" ", " ");
19            System.out.print("M: "+m.group(2)+" ", " ");
20            System.out.print("Y: "+m.group(3)+"'\n'");
21        }
22
23        m = Pattern.compile(pat2).matcher(str);
24        System.out.println("Named groups");
25        while (m.find()) {
26            System.out.println(m.groupCount()+" groups:");
27            System.out.print("D: "+m.group("day")+" ", " ");
28            System.out.print("M: "+m.group("month")+" ", " ");
29            System.out.print("Y: "+m.group("year")+"'\n'");
30        }
31    }
32 }
```

which prints

```

Unnamed groups
3 groups:
D: 12, M: 07, Y: 2014
3 groups:
D: 6, M: 6, Y: 2010
3 groups:
D: 1, M: 11, Y: 2011
Named groups
3 groups:
D: 12, M: 07, Y: 2014
3 groups:
D: 6, M: 6, Y: 2010
3 groups:
D: 1, M: 11, Y: 2011

```

Inside a regex, we can refer to the previously found groups using `\n` expressions, where `n` is the number of the group we are interested in (this is called *backreference*). In the following example we want to find fragments of a text enclosed in apostrophes or double quotes. We thus look for either an apostrophe or a double quote, but we have to ensure that the closing quote is of the same type as the opening one:

```

String reg = "(['\"])[^'\"]*\\1";
String text = "'abc' xx \"def\" yy \"ghi' zz";
Matcher m = Pattern.compile(reg).matcher(text);
while (m.find())
    System.out.println(m.group());

```

The program prints

```

'abc'
"def"

```

Note that the sequence `ghi` is enclosed in non-matching quotes, so, correctly, it has not been found.

Backreferences can also be used in replacement texts when using `String.replaceAll` method. There is also such a method in class `Matcher` and in fact

```
str.replaceAll(regex, text)
```

is equivalent to

```
Pattern.compile(regex).matcher(str).replaceAll(text)
```

For example, suppose we have a file with full names in the order first name and last name separated by at least one space, but what we want is to have them in reversed order: first the last name and then the first name separated by exactly one space. In the replacement text, we refer to groups using `$n` notation, where `n` is the group number, as in the example below:

```

String orig = "John Smith, Mary Brown";
String modif = orig.replaceAll(
    "(\\p{L}+)\\s+(\\p{L}+)", "$2 $1");
System.out.println("Orig : " + orig + '\n' +
    "Modif: " + modif);

```

The program prints

```
Orig : John  Smith, Mary  Brown
Modif: Smith John, Brown Mary
```

We could have used named groups instead; then in the replacement string we refer to groups using `${name}` notation:

```
String orig  = "John  Smith, Mary  Brown";
String modif = orig.replaceAll(
    "(?<first>\\p{L}+)\\s+(?<last>\\p{L}+)",
    "${last} ${first}");
System.out.println("Orig : " + orig + '\n' +
    "Modif: " + modif);
```

with the result as before.

1.5 Option flags

There are several options which influence the process of compilation of a pattern. They can be specified in two ways: as an additional argument to **compile** or embedded directly into the regex. Options are defined in class **Pattern** as named static integer fields, which may be ORed, if we want several of them. If we choose to specify them embedded in our regex, we do it by including expression `(?<letters>)` where `<letters>` can be one or more letters denoting different options. For example, suppose we want to turn on the options DOTALL and MULTILINE; we can do it like this

```
import static java.util.regex.Pattern.*;
// ...
String regex = "...";
Pattern p = Pattern.compile(regex, DOTALL | MULTILINE);
```

or like this

```
String regex = "(?sm)...";
Pattern p = Pattern.compile(regex);
```

as the letter 's' denotes DOTALL ('s' because it's called 'single-line' in Perl) and 'm' stands for MULTILINE. Note that enabling options usually induces some performance penalty, so don't do it if it's not necessary.

Some of the most useful options are described below (there are others).

1.5.1 MULTILINE

This option enables the so called 'multi-line mode', which means that expressions `^` and `$` match just at the beginning of each a line and just at the and of each line, respectively. Normally, these symbols match only at the beginning and at the end of the whole input text; in multi-line mode those are still available as `\A` and `\Z`.

This option can also be enabled by embedding the flag `(?m)`.

For example, the following program

```
String s = "A 123\nD 456";
Matcher m1 = Pattern.compile("^\\w").matcher(s);
Matcher m2 = Pattern.compile("(?m)^\\w").matcher(s);
System.out.print("m1 : ");
```

```

while(m1.find())
    System.out.print(m1.group() + " ");
System.out.print("\nm2 : ");
while(m2.find())
    System.out.print(m2.group() + " ");
System.out.println();

```

will print

```

m1 : A
m2 : A D

```

because in the first case we only looked for a letter at the beginning of the entire input, while in the second case — at the beginning of each line separately.

1.5.2 DOTALL

This option changes the interpretation of the dot (`.`). By default, it denotes ‘any character except new line’, while with this option enabled a dot matches any character, *including* the new line.

This option can be enabled by embedding the flag `(?s)`.

For example, the following program

```

String s = "A 123\n456 B";
boolean b1 = Pattern.compile(
    "\\w.*\\w").matcher(s).matches();
boolean b2 = Pattern.compile(
    "(?s)\\w.*\\w").matcher(s).matches();
System.out.println("b1=" + b1 + "; b2=" + b2);

```

will print

```

b1=false; b2=true

```

because in the first case `.*` consumes everything up to the new line (but not any further) and there is no letter at the end, while in the second case everything, including the new line character, will be consumed by `.*` reaching the final letter ‘B’.

1.5.3 CASE_INSENSITIVE and UNICODE_CASE

Enabling the `CASE_INSENSITIVE` option makes the lower- and uppercase letters indistinguishable. However,

this works for Latin letters only. If it should apply to non-Latin letters as well, we have to enable additionally the option `UNICODE_CASE`.

The options can be enabled by embedding the flags `(?i)` and `(?u)`, respectively, or we can enable both of them by `(?iu)`.

For example, the following program

```

String s = "PaRiS";
boolean b1 = Pattern.compile(
    "paris").matcher(s).matches();
boolean b2 = Pattern.compile(
    "paris", Pattern.CASE_INSENSITIVE)
    .matcher(s).matches();
System.out.println("b1=" + b1 + "; b2=" + b2);

```

will print

```
b1=false; b2=true
```

as in the second case we ignore the case of letters, so `PaRiS` and `paris` are considered equivalent.

Below, we illustrate the fact that `CASE_INSENSITIVE` (or `(?i)`) will not work non-Latin letters – we need to add `UNICODE_CASE` (or `(?u)`):

```
jshell> "Łł".matches("(?i)(\\p{L})\\1")
$1 ==> false

jshell> "Łł".matches("(?iu)(\\p{L})\\1")
$2 ==> true
```

1.5.4 `UNICODE_CHARACTER_CLASS`

When this option is enabled, some classes of characters, which normally apply only to ASCII characters, take into account all Unicode characters.

The options can be enabled by embedding the flags `(?U)` (note the *capital* U). Enabling it implies also `UNICODE_CASE`.

For example, the following program

```
String s = "Żółć";
boolean b1 = Pattern.compile(
    "\\w+").matcher(s).matches();
boolean b2 = Pattern.compile(
    "\\w+", Pattern.UNICODE_CHARACTER_CLASS)
    .matcher(s).matches();
System.out.println("b1=" + b1 + "; b2=" + b2);
```

will print

```
b1=false; b2=true
```

as in the second case `\w` matches also non-ASCII letters.

1.6 Some examples

In the following example we look for names in file *input.txt* (in UTF-8 encoding): we make a rather naïve assumption that a string with the first letter in uppercase and the remaining characters in lowercase must be a name... Then we prints a list of names together with numbers of occurrences and lists of line numbers where a given name appeared. We use an auxiliary class `Name`

Listing 3

GXR-RegExNames/Name.java

```
1 import java.util.ArrayList;
2
3 public class Name {
4     private String name;
5     private ArrayList<Integer> list;
6     private int total = 0;
```

```

7
8     public Name(String name, int lineNo) {
9         this.name = name;
10        list = new ArrayList<Integer>();
11        list.add(lineNo);
12        total = 1;
13    }
14
15    public void addNum(int lineNo) {
16        if (list.get(list.size()-1) != lineNo)
17            list.add(lineNo);
18        ++total;
19    }
20
21    @Override
22    public String toString() {
23        return name + " (" + total + ") in lines " + list;
24    }
25 }

```

and the program might look like this:

Listing 4

GXR-RegExNames/RegEx.java

```

1 import java.io.BufferedReader;
2 import java.io.IOException;
3 import java.nio.file.Files;
4 import java.nio.file.Path;
5 import java.nio.file.Paths;
6 import java.util.Map;
7 import java.util.TreeMap;
8 import java.util.regex.Matcher;
9 import java.util.regex.Pattern;
10 import java.util.regex.PatternSyntaxException;
11 import static java.nio.charset.StandardCharsets.UTF_8;
12
13 /* Reads file "input.txt" (in UTF-8) and selects
14 * names (strings starting with capital letter with
15 * other letters in lowercase. Then prints a list of
16 * names together with numbers of occurrences and lists
17 * of line numbers where a given name appeared.
18 * Uses class Name.
19 */
20 public class RegEx {
21
22     private Map<String,Name> map;
23
24     public static void main(String[] args) {
25         new RegEx("input.txt");

```



```

26     System.exit(0);
27 }
28
29 public RegEx(String fileName) {
30     Path filein = Paths.get(fileName);
31     if (!Files.exists(filein) ||
32         !Files.isReadable(filein) ||
33         Files.isDirectory(filein)) {
34         System.err.println("Invalid input file !!!");
35         System.exit(1);
36     }
37     // File input = new File(filename);
38     // BufferedReader br = null;
39     map = new TreeMap<String,Name>();
40     try (
41         BufferedReader br =
42             Files.newBufferedReader(filein, UTF_8)
43     ) {
44         String line, name,
45             patt = "\\b\\p{Lu}\\p{Ll}+\\b";
46         Pattern pattern = null;
47         try {
48             pattern = Pattern.compile(patt);
49         } catch (PatternSyntaxException e) {
50             System.err.println("Wrong pattern");
51             System.exit(1);
52         }
53         Matcher matcher = pattern.matcher("");
54
55         int lineNo = 0;
56         while ((line = br.readLine()) != null) {
57             lineNo++;
58             matcher.reset(line);
59             if (matcher.find()) {
60                 do {
61                     name = matcher.group();
62                     if (!map.containsKey(name))
63                         map.put(name,
64                             new Name(name,lineNo));
65                     else
66                         map.get(name).addNum(lineNo);
67                 } while (matcher.find());
68             }
69         }
70     } catch (IOException e) {
71         System.err.println("Something wrong - exiting");
72         e.printStackTrace();
73         System.exit(1);
74     }
75 }

```

```

76         for (Map.Entry<String,Name> e : map.entrySet())
77             System.out.println(e.getValue());
78     }
79 }

```

The program below asks for a regex and a text (in a loop); it then prints information about matches found:

Listing 5

GXU-Regexes/Regexes.java

```

1  import java.io.Console;
2  import java.util.regex.Matcher;
3  import java.util.regex.Pattern;
4
5  public class Regexes {
6      public static void main(String[] args){
7          Console console = System.console();
8          if (console == null) {
9              System.err.println("Console unavailable");
10             System.exit(1);
11         }
12         while (true) {
13             String reg = console.readLine(
14                 "%nRegex ('q' to quit) -> ");
15             if ("q".equals(reg)) return;
16             String inp = console.readLine(
17                 "Input string -> ");
18             Pattern pattern = Pattern.compile(reg);
19             Matcher matcher = pattern.matcher(inp);
20
21             boolean found = false;
22             while (matcher.find()) {
23                 found = true;
24                 console.format("Found '%s' at %d-%d.%n",
25                     matcher.group(),
26                     matcher.start(),
27                     matcher.end());
28             }
29             if(!found){
30                 console.format("No match found.%n");
31             }
32         }
33     }
34 }

```

and a similar program with a graphical interface:

Listing 6

GXW-REExplorer/REExplorer.java

```
1  import java.awt.BorderLayout;
2  import java.awt.FlowLayout;
3  import java.awt.Font;
4  import java.awt.event.ActionEvent;
5  import java.util.regex.Matcher;
6  import java.util.regex.Pattern;
7  import java.util.regex.PatternSyntaxException;
8  import javax.swing.AbstractAction;
9  import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JDialog;
12 import javax.swing.JFrame;
13 import javax.swing.JLabel;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingUtilities;
19 import static javax.swing.JDialog.DISPOSE_ON_CLOSE;
20 import static javax.swing.JFrame.EXIT_ON_CLOSE;
21
22 public class REExplorer {
23
24     public static void main(String[] args) {
25         new REExplorer();
26     }
27
28     private REExplorer() {
29         final JFrame f = new JFrame("RE Explorer");
30         f.setDefaultCloseOperation(EXIT_ON_CLOSE);
31
32         final JLabel labx = new JLabel("REGEX:");
33         final JTextField rege = new JTextField(40);
34         final JButton gobu = new JButton("Go!");
35         labx.setFont(new Font("Dialog",Font.PLAIN,18));
36         gobu.setFont(new Font("Dialog",Font.PLAIN,18));
37         rege.setFont(new Font("Dialog",Font.PLAIN,18));
38         rege.setBorder(BorderFactory.
39             createEmptyBorder(5,5,5,5));
40         JPanel pans = new JPanel();
41         pans.setLayout(new FlowLayout());
42         pans.add(labx);
43         pans.add(rege);
44         pans.add(gobu);
45
46         final JTextArea text = new JTextArea(10,40);
47         text.setFont(new Font("Dialog",Font.PLAIN,18));
48         text.setBorder(BorderFactory.
```

```

49         createTitledBorder(
50             "Enter text to be searched below"));
51
52     AbstractAction act = new AbstractAction() {
53         @Override
54         public void actionPerformed(ActionEvent e) {
55             showText(getMatches(rege.getText(),
56                             text.getText()));
57         }
58     };
59     rege.addActionListener(act);
60     gobu.addActionListener(act);
61
62     JScrollPane scroll = new JScrollPane(text,
63         JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED,
64         JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED);
65
66     JPanel panel = new JPanel();
67     panel.setLayout(new BorderLayout());
68     panel.add(scroll, BorderLayout.CENTER);
69     panel.add(pans, BorderLayout.SOUTH);
70
71     f.setContentPane(panel);
72
73     SwingUtilities.invokeLater(new Runnable() {
74         public void run() {
75             f.pack();
76             f.setLocationRelativeTo(null);
77             f.setVisible(true);
78         }
79     });
80 }
81
82 private String getMatches(String regex, String text) {
83     StringBuilder sb = new StringBuilder(200);
84     String nL = System.getProperty("line.separator");
85
86     sb.append("Pattern: \"" + regex + "\"" + nL +
87         "-- Text from here -----" + nL + text +
88         nL + "-- to here -----" + nL + nL);
89     // Compiling the pattern
90     Pattern pattern = null;
91     try {
92         pattern = Pattern.compile(regex);
93     } catch (PatternSyntaxException exc) {
94         sb.append(
95             "Error: " + exc.getMessage() + nL);
96         return sb.toString();
97     }
98

```

```

99     Matcher matcher = pattern.matcher(text);
100
101     // does the whole text match the pattern?
102     boolean match = matcher.matches();
103     sb.append("matches() gives: " +
104             (match ? "YES" : " NO") + nL + nL);
105     // groups (if any)
106     if (match) {
107         int gr = matcher.groupCount();
108         sb.append(gr + " groups:" + nL);
109         for (int i = 1; i <= gr; ++i)
110             sb.append("  " + i + ": " +
111                     matcher.group(i) + nL);
112         sb.append(nL);
113     }
114
115     matcher.reset();
116
117     // looking for matches inside the text
118     boolean found = matcher.find();
119     if (!found)
120         sb.append("find() didn't find anything" + nL);
121     else
122         do {
123             sb.append("find() found \"" +
124                     matcher.group() + "\" at " +
125                     (matcher.start()+1) + "-" +
126                     matcher.end() + nL);
127         } while(matcher.find());
128
129     return sb.toString();
130 }
131
132 private void showText(String text) {
133     final JDialog dg = new JDialog();
134     dg.setDefaultCloseOperation(DISPOSE_ON_CLOSE);
135     JTextArea area = new JTextArea(20,45);
136     area.setText(text);
137     area.setBorder(BorderFactory.
138             createEmptyBorder(10,5,10,5));
139     area.setFont(new Font("Monospaced",Font.PLAIN,20));
140     area.setEditable(false);
141     dg.add(new JScrollPane(area));
142     SwingUtilities.invokeLater(new Runnable() {
143         public void run() {
144             dg.pack();
145             dg.setLocationRelativeTo(null);
146             dg.setVisible(true);
147         }
148     });

```

```

149     }
150 }

```

Another example:

Listing 7

GXY-RegFind/RegFind.java

```

1  import java.util.regex.Matcher;
2  import java.util.regex.Pattern;
3
4  public class RegFind {
5      public static void main (String[] args) {
6
7          spl("Joe : Mary::Jane", ":");
8          spl("Joe : Mary:Jane", "\\s*:\\s*");
9
10         mat("Joe : Mary:Jane", "[^:]*(:[~:~]*){2}");
11         mat("123 xxx ABCD", "\\.*\\d+[^A-Z]*[A-Z]{3,}");
12         mat("Jane Crawford", "\\w+\\s+([A-Z]\\.)?\\s*\\w+");
13
14         rep("a  b  c", "\\s+", " ");
15
16         process("kot\\b",
17             "Kot kot kotek");
18         process("(kot)",
19             "Kot kot kotek");
20         process("\\b[A-Z] [a-z]+\\b",
21             "cat Dog hen Cow aHorse Z");
22         process("^.*(kot).*(tek)",
23             "Kot kot kotek");
24         process("(a.*)((w).*)",
25             "a b c d c b x");
26         process("(\\d{1,3}\\.){3}\\d{1,3}",
27             "1.1.1.2 12.12.34.231 234 xx 3.21.21.21 zz");
28     }
29
30     private static void process(String reg, String str) {
31         Pattern p = Pattern.compile(reg);
32         Matcher m = p.matcher(str);
33         System.out.println("== match and find =====");
34         System.out.println("STRING : " + str);
35         System.out.println("REGEX : " + reg);
36         boolean gr = m.matches();
37         System.out.println("Matches: " + gr);
38         if (gr) {
39             System.out.println("Groups : " + m.groupCount());
40             for (int i = 1; i <= m.groupCount(); ++i) {
41                 System.out.println("Group " + i +
42                     " = '" + m.group(i) + "'");

```

```
43     }
44 }
45 m.reset();
46 while (m.find()) {
47     System.out.println(
48         "Found " + ": " + m.group() +
49         " at " + m.start() + "-" + m.end());
50 }
51 }
52
53 private static void spl(String str, String reg) {
54     System.out.println("== split =====");
55     System.out.println("STRING : " + str);
56     System.out.println("REGEX  : " + reg);
57     String[] s = str.split(reg);
58     System.out.print(s.length + " terms:");
59     for (int i = 0; i < s.length; ++i)
60         System.out.print(" '" + s[i] + "'");
61     System.out.println();
62 }
63
64 private static void mat(String str, String reg) {
65     System.out.println("== matches =====");
66     System.out.println("STRING : " + str);
67     System.out.println("REGEX  : " + reg);
68     System.out.println(str.matches(reg));
69 }
70
71 private static void rep(String str, String reg, String with) {
72     System.out.println("== replace =====");
73     System.out.println("STRING : " + str);
74     System.out.println("REGEX  : " + reg);
75     System.out.println("WITH   : '" + with + "'");
76     System.out.println(str.replaceAll(reg,with));
77 }
78 }
```

Section 2

Basic data structures

Computer programs are all about processing information. The amount of information that has to be handled is very often really huge and grows rapidly from year to year. Therefore, it is very important to organize the data in such a way that processing it is fast and efficient. Generations of scientists have been working (and are still working) on that problem. Generally, designing a way in which the data is structurized is related to the way in which it will be used; both aspects are studied by the branch of computer science called Algorithms and Data Structures.

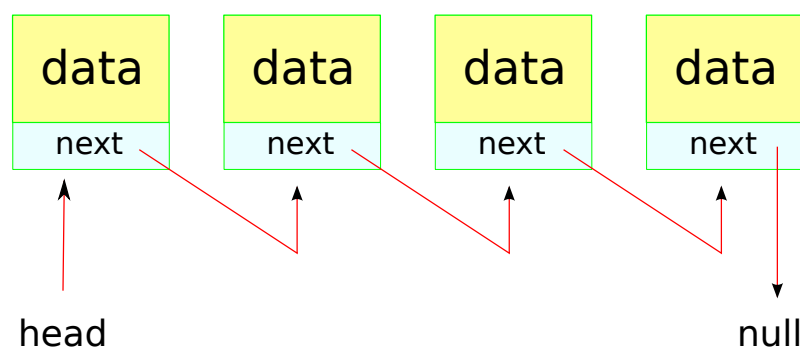
One important, and very useful, data structure is already known to us: arrays. Representing data in the form of arrays has many advantages, perhaps the most important being the speed of access to individual elements (using indices), which is “immediate” ($\mathcal{O}(1)$) and does not depend on the size of the array. The downside, however, is the fact that the size of arrays must be fixed at their creation and no elements can be then “deleted” or added.

In this section, we will mention just a few most fundamental data structures based on lists (ordered sequences of elements): lists as such, queues and stacks. Of course, there are many other equally useful structures – like dictionaries (maps), trees, graphs etc. — you will study them later, although we will say a few words about maps in the section on collections in Java.

2.1 Singly linked lists

A singly linked list represents a sequence of pieces of data of a certain type (in an extreme case, references to object of type **Object** which may represent anything). Each such piece of data is “wrapped” in an object of some type (conventionally called **Node**) as its field, the other field being the reference to the next node of the list. In this way, we can build a chain of nodes, where each node contains information about the next. We need some kind of a marker which will mark the last node as being the last: for example, we can adopt the convention that the last node’s **next** field is **null**.

Thus, the situation looks like this:



Note that having the reference to the first node (conventionally called **head**), we can access all the nodes, because in each of them we will find the reference to the next one.

Let us consider an example. The class **Node** represents a node containing data – in this case just an **int** – and a reference to the next node:

Listing 8

BGU-SingList/Node.java

```
1  // could be private static inner class of MyList
2  public class Node {
3      int data;
4      Node next;
5      Node(int data, Node next) {
6          this.data = data;
7          this.next = next;
8      }
9      Node(int data) {
10         this(data,null);
11     }
12 }
```

The class representing the whole list will contain only one field: the head of the list, i.e., the reference to its first node. We will add methods which add new nodes at the beginning (**addFront**) and at the end (**addBack**) of the list. Notice that adding a node at the end requires traversing the whole list. Also, to count elements of the list (the **size** method), we have to traverse the whole list.

The **showListReversed** method prints the list in the reverse order by calling private, *recursive* function **showRev**. The main idea here is that the function *first* calls itself for the next node (if it exists), and only then, when the flow of control returns to it, prints information on *this* element.

Listing 9

BGU-SingList/MyList.java

```
1  public class MyList {
2      private Node head;
3
4      public MyList() {
5          head = null;
6      }
7
8      public void addFront(int data) {
9          head = new Node(data,head);
10     }
11
12     public void addBack(int data) {
13         if (head == null) {
14             addFront(data);
15             return;
16         }
17         Node tmp = head;
18         while(tmp.next != null)
19             tmp = tmp.next;
20         tmp.next = new Node(data);
21     }
22     public void showList() {
23         System.out.print("[ ");
```

```

24     Node tmp = head;
25     while(tmp != null) {
26         System.out.print(tmp.data + " ");
27         tmp = tmp.next;
28     }
29     System.out.println("");
30 }
31 public void showListReversed() {
32     System.out.print("[ ");
33     showRev(head);
34     System.out.print("]");
35 }
36 private void showRev(Node h) {
37     if (h.next != null) showRev(h.next);
38     System.out.print(h.data + " ");
39 }
40 public int size() {
41     // inefficient!
42     int count = 0;
43     Node tmp = head;
44     while(tmp != null) {
45         ++count;
46         tmp = tmp.next;
47     }
48     return count;
49 }
50 public boolean empty() {
51     return head == null;
52 }
53 }

```

In **Main**, we build a list and then print its contents

Listing 10

BGU-SingList/Main.java

```

1 public class Main {
2     public static void main(String[] args) {
3         MyList list = new MyList();
4         list.addBack(4);
5         list.addBack(5);
6         list.addFront(3);
7         list.addFront(2);
8         list.addFront(1);
9         list.showList();
10        list.showListReversed();
11        System.out.println("\nsize = " + list.size());
12    }
13 }

```

The program prints

```
[ 1 2 3 4 5 ]
[ 5 4 3 2 1 ]
size = 5
```

Of course, for our list to be useful, we would have to add more methods: for example to insert new node in the middle of the list, to remove its elements etc.

2.2 Stacks

A **stack** is a very simple but extremely useful data structure. It provides three operations

- **push** — adding new element;
- **pop** — removing one element (and getting its value);
- **empty** — checking if the stack is empty.

However, adding (pushing) and removing (popping) elements must be organized in such the way that what we pop is always what we pushed most recently. For example, if we push first *A*, then *B* and then *C*, three consecutive “pops” will give us first *C*, then *B* and lastly *A*. A structure organized in this way is called **LIFO** (*last in, first out*).

A stack may be easily implemented as a singly linked list: we always add (push) an element at the front (so it becomes the new head) and pop elements also from the front. Note that traversing the list will never be necessary!

Let us consider an example. Wrapping class **Node** looks the same as before

Listing 11

BGW-Stack/Node.java

```
1  // could be private static inner class of MyStack
2  public class Node {
3      private int data;
4      private Node next;
5      Node(int data, Node next) {
6          this.data = data;
7          this.next = next;
8      }
9      Node(int data) {
10         this(data,null);
11     }
12     int  getData() { return data; }
13     Node getNext() { return next; }
14 }
```

and the implementation is very simple

Listing 12

BGW-Stack/MyStack.java

```
1  public class MyStack {
2      private Node head = null;
```

```

3     public void push(int data) {
4         head = new Node(data,head);
5     }
6     public int pop() {
7         int d = head.getData();
8         head = head.getNext();
9         return d;
10    }
11    public boolean empty() {
12        return head == null;
13    }
14 }

```

In **Main**, we test our stack:

Listing 13

BGW-Stack/Main.java

```

1 public class Main {
2     public static void main(String[] args) {
3         MyStack st = new MyStack();
4         st.push(4);
5         st.push(5);
6         st.push(6);
7         while (!st.empty())
8             System.out.println("popping " + st.pop());
9     }
10 }

```

The program prints

```

popping 6
popping 5
popping 4

```

The output shows that, indeed, what we have pushed last (value 6) has been popped first.

Another popular implementation of the stack is based on an array. This is illustrated in the program below. In the constructor, we allocate an array (with fixed size) and a field **top** which corresponds to the index of the first free ‘slot’ in the array (0 at the beginning). When pushing a new value, we insert it into the array at the position **top** and we increment **top** by 1. When popping, we first decrement **top** and we return the value from this position (which will then be treated as the first free ‘slot’). The downside of this implementation is the fact that the capacity of the stack will be limited.

Listing 14

BGZ-ArrStack/ArrStack.java

```

1 public class ArrStack {
2     private int[] elems;

```

```

3     private int top = 0;
4     public ArrStack(int maxSize) {
5         elems = new int[maxSize];
6     }
7     public void push(int e) {
8         // throws if stack full
9         elems[top++] = e;
10    }
11    public int pop() {
12        // throws if stack empty
13        return elems[--top];
14    }
15    public boolean empty() {
16        return top == 0;
17    }
18 }
19

```

In **Main**, we create the stack and test its behavior:

Listing 15

BGZ-ArrStack/Main.java

```

1 public class Main {
2     public static void main (String[] args) {
3         ArrStack stack = new ArrStack(5);
4         for (int i = 1; i <= 5; ++i)
5             stack.push(i*i);
6         while (!stack.empty())
7             System.out.print(stack.pop() + " ");
8         System.out.println();
9     }
10 }

```

and we get

```
25 16 9 4 1
```

Note that we do *not* handle the situations when an attempt is made to pop from an empty stack or to push onto a full stack: an exception will be thrown anyway, as an out-of-bounds index will then be used.

2.3 Queues

A **queue** is similar to a stack, but with the reversed order of “pops”: now, what we pop is not the most recent element pushed, but the “oldest” — this is called **FIFO** (*first in, first out*). For example, if we enqueue first *A*, then *B* and then *C*, three consecutive “dequeues” will give us first *A*, then *B* and finally *C*.

A very simple implementation is again based on a singly linked list: we always add elements at the end and remove them from the beginning. This will not require traversing

the list, if we always keep the reference to the last element (called **tail** in the example below).

Let us consider an example. The **Node** class is now a nested class: it is declared as static and *private* class *inside* the **MyQueue** class, so it will be invisible for the user but can be used inside the enclosing class **MyQueue** (we could have done it in previous examples as well):

Listing 16

BGY-SimpQueue/MyQueue.java

```

1 public class MyQueue {
2     // nested class
3     private static class Node {
4         String data;
5         Node next = null;
6         Node(String d) { data = d; }
7     }
8
9     private Node head, tail;
10
11    public MyQueue() {
12        head = tail = null;
13    }
14    public void enqueue(String s) {
15        if (head == null)
16            head = tail = new Node(s);
17        else
18            tail = tail.next = new Node(s);
19    }
20    public String dequeue() {
21        String s = head.data;
22        if ((head = head.next) == null) tail = null;
23        return s;
24    }
25    public boolean empty() {
26        return head == null;
27    }
28 }

```

In **Main**, we test our queue:

Listing 17

BGY-SimpQueue/Main.java

```

1 public class Main {
2     public static void main (String[] args) {
3         MyQueue qS = new MyQueue();
4         for (double d = 0.5; d < 5; d += 1)
5             qS.enqueue("" + d);
6         while (!qS.empty())
7             System.out.print(qS.dequeue() + " ");

```

```
8         System.out.println();  
9     }  
10 }
```

the program prints

0.5 1.5 2.5 3.5 4.5

so the elements are printed in the same order in which they have been enqueued.

Section 3

Abstract classes, interfaces, inner classes and lambdas

3.1 Abstract classes

Abstract classes are those which are declared as such with the keyword **abstract**. They usually (although it's not obligatory) contain at least one abstract method, i.e., a method only declared but not defined — it also has to be marked as **abstract**. It is not possible to create an object of an abstract class (even if it *is* fully implemented) — their *raison d'être* is to be extended. A classes which extends (inherits from) an abstract class must provide *definitions* of methods declared but not defined in its abstract superclass, otherwise it will still be abstract itself.

Abstract classes provide a common interface (collection of methods) to a set of classes which can implement these methods in different ways.

What is important is that we can declare references to abstract types while objects pointed to by these references are all of derived types (they cannot be objects of the base, abstract, class because such object cannot be even created).

Let us consider an example of an abstract class **Figure**, representing geometrical figures. Of course, if all we know is that something is a figure, we cannot say much about its area or perimeter: for this we should know its type more precisely. Therefore **getArea** and **getPerimeter** are declared abstract:

Listing 18

EPZ-AbsFig/Figure.java

```
1 abstract class Figure {
2
3     abstract public double getArea();
4     abstract public double getPerimeter();
5
6     static public Figure getFigMaxArea(Figure[] figs) {
7         double maxarea = 0;
8         Figure maxfig = null;
9         for (Figure f : figs) {
10             double area = f.getArea();
11             if (area > maxarea) {
12                 maxarea = area;
13                 maxfig = f;
14             }
15         }
16         return maxfig;
17     }
18
19     @Override
20     public String toString() {
21         return " area: "
```



```

22         + String.format("%6.3f",getArea())
23         + "; perimeter: "
24         + String.format("%6.3f",getPerimeter());
25     }
26 }

```

Note that the class *defines* the static method **getFigMaxArea** taking an array of references of type **Figure**. Object referred to by elements of these array will never be of type **Figure** because such object cannot be even created. But their type has to extend **Figure** and hence, to be concrete, provide definitions of all abstract methods. Therefore, due to polymorphism, we can invoke on them the methods **getArea** and **getPerimeter**.

Class **Circle** inherits from **Figure** and provides definitions of the abstract methods. It also overrides **toString** — note, how its implementation uses that from the base class (by using **super**).

Listing 19

EPZ-AbsFig/Circle.java

```

1  public class Circle extends Figure {
2      private double r;
3
4      public Circle(double r) {
5          this.r = r;
6      }
7
8      @Override
9      public double getArea() {
10         return Math.PI*r*r;
11     }
12     @Override
13     public double getPerimeter() {
14         return 2*Math.PI*r;
15     }
16     @Override
17     public String toString() {
18         return "Circle    (    r=" + r + "    )"
19             + super.toString();
20     }
21 }

```

as does **Rectangle**

Listing 20

EPZ-AbsFig/Rectangle.java

```

1  public class Rectangle extends Figure {
2      private double a, b;
3
4      public Rectangle(double a, double b) {

```

```

5         this.a = a;
6         this.b = b;
7     }
8
9     @Override
10    public double getArea() {
11        return a*b;
12    }
13
14    @Override
15    public double getPerimeter() {
16        return 2*(a+b);
17    }
18
19    @Override
20    public String toString() {
21        return "Rectangle (a=" + a + " b=" + b + ")"
22            + super.toString();
23    }
24 }

```

and we can now check our classes in **main**

Listing 21

EPZ-AbsFig/Main.java

```

1 import java.util.Locale;
2
3 public class Main {
4     public static void main(String[] args) {
5         // to have decimal point instead of a comma...
6         Locale.setDefault(Locale.US);
7
8         Figure[] figs = {
9             new Circle(2),      new Rectangle(9,1),
10            new Rectangle(4,3), new Circle(4)
11        };
12
13        Figure fig = Figure.getFigMaxArea(figs);
14        System.out.println("\nLargest area: \n" + fig);
15    }
16 }

```

which prints

```

Largest area:
Circle    (    r=4.0    ) area: 50.265; perimeter: 25.133

```

3.2 Interfaces

Interfaces are, in a sense, “pure” abstract classes — they only declare one or more methods, but do not implement them (but see below). All methods are, by definition, **public**, even if not declared as such. Also, unimplemented methods are *not* declared as **abstract** (although they are abstract).

Since Java 8, interfaces can also *define* (not only declare):

- final, static constants (these were supported also in earlier versions of Java)
- default methods, marked with the keyword **default**;
- *private* methods, used internally by its default methods — these private methods, of course, are not visible for the user and do not belong to the ‘contract’;
- *static* methods — which can be public and are accessible for the users.

Interfaces with *only one* abstract method (but, perhaps, with some default methods and static member functions already implemented) are called **functional interfaces** and play a special rôle in Java (more about it later — see 9, p. 112).

What’s important is the fact that any class can *extend* (directly inherit from) only one class, but may *implement* **any number of interfaces**. Also records (see Sec. ??, p. ??) can implement interfaces.

Interfaces define a “contract” — if we know that a class implements a given interface, we know that all its abstract methods must have been implemented somehow and hence it is safe to call these methods on object of such a class. We can also declare references of interface type — they can refer to objects of *any* classes implementing the interface, even if their types belong to completely separate subtrees of the class hierarchy.

Let us consider the following example. We define an interface, **IMyStack**, which defines a ‘contract’ of a stack. What is a stack? It is something that has **pop**, **push** and **empty** methods. We can then create concrete classes that implement this interface in different ways — for example using an array or using the singly-linked list (**MyStackArr** and **MyStackList**, respectively, in the example below). Note the function **testStack**. Its single argument is declared as “something” of type **IMyStack** — there will be no objects of this type because it is not a concrete class but an interface. However, this declaration means “anything what implements **IMyStack** will be accepted”. We can then safely call **push**, **pop** and **empty** on the object passed to the function because we know that they have to be implemented somehow. For example, the program below:

Listing 22

ELJ-InterStack/MyStacks.java

```

1 public class MyStacks {
2     public static void main(String[] args) {
3         testStack(new MyStackArr(10));
4         testStack(new MyStackList());
5     }
6
7     public static void testStack(IMyStack stack) {
8         stack.push(5);
9         stack.push(4);
10        stack.push(3);
11        stack.push(2);
12        while (!stack.empty()) {
13            System.out.print(stack.pop() + " ");

```

```
14     }
15     System.out.println();
16 }
17 }
18
19
20 interface IMyStack {
21     int pop();           // public automatically
22     void push(int i);
23     boolean empty();
24 }
25
26 class MyStackArr implements IMyStack {
27     int[] arr;
28     int top;
29     public MyStackArr(int size) {
30         arr = new int[size];
31         top = 0;
32     }
33     public int pop() {
34         return arr[--top];
35     }
36     public void push(int i) {
37         arr[top++] = i;
38     }
39     public boolean empty() {
40         return top == 0;
41     }
42 }
43
44 class MyStackList implements IMyStack {
45     private static class Node {
46         int data;
47         Node next;
48         Node(int d, Node n) {
49             data = d;
50             next = n;
51         }
52         Node(int d) {
53             this(d, null);
54         }
55     }
56     private Node head = null;
57     public int pop() {
58         int d = head.data;
59         head = head.next;
60         return d;
61     }
62     public void push(int d) {
63         head = new Node(d, head);
```

```

64     }
65     public boolean empty() {
66         return head == null;
67     }
68 }

```

prints

```

2 3 4 5
2 3 4 5

```

Let us now consider another example. We define an interface that has only one abstract method: **lt** (*less than*). All other methods (**gt** — *greater than*, **ge** — *greater or equal than*, etc.) have default implementation expressed, directly or indirectly, by this one abstract method. Therefore, to implement this interface we only have to implement **lt** — all other methods will then work automatically!

Listing 23

ELS-DefIntSimple/MyCompar.java

```

1  public interface MyCompar {
2      // abstract
3      boolean lt(int lhs, int rhs);
4
5      // implemented directly in terms of the abstract
6      default boolean gt(int lhs, int rhs) {
7          return lt(rhs, lhs);
8      }
9      default boolean ge(int lhs, int rhs) {
10         return !lt(lhs, rhs);
11     }
12     default boolean le(int lhs, int rhs) {
13         return !lt(rhs, lhs);
14     }
15     default boolean eq(int lhs, int rhs) {
16         return !lt(lhs, rhs) && !lt(rhs, lhs);
17     }
18     // implemented indirectly in terms of the abstract
19     default boolean ne(int lhs, int rhs) {
20         return !eq(lhs, rhs);
21     }
22 }

```

Then we define three classes implementing this interface: one compares integers by their values

Listing 24

ELS-DefIntSimple/CompVal.java

```

1  public class CompVal implements MyCompar {
2      @Override
3      public boolean lt(int lhs, int rhs) {

```

```

4         return lhs < rhs;
5     }
6 }

```

the second by sum of digits

Listing 25

ELS-DefIntSimple/CompDigits.java

```

1 public class CompDigits implements MyCompar {
2     @Override
3     public boolean lt(int lhs, int rhs) {
4         return sumOfDigs(lhs) < sumOfDigs(rhs);
5     }
6     private static int sumOfDigs(int n) {
7         int sum = 0;
8         n = n < 0 ? -n : n;
9         while (n != 0) {
10             sum += n % 10;
11             n /= 10;
12         }
13         return sum;
14     }
15 }

```

and the third by value, but reversed

Listing 26

ELS-DefIntSimple/CompValRev.java

```

1 public class CompValRev implements MyCompar {
2     @Override
3     public boolean lt(int lhs, int rhs) {
4         return lhs > rhs;
5     }
6 }

```

Now in **Main** we can use all three implementations; each of them has full set of all methods implemented although they override only one method:

Listing 27

ELS-DefIntSimple/Main.java

```

1 public class Main {
2     public static void main (String[] args) {
3
4         MyCompar cmpVal = new CompVal();
5         MyCompar cmpSum = new CompDigits();
6         MyCompar cmpVaR = new CompValRev();
7
8         compare("cmpVal - BY VALUE", cmpVal,

```

```

9         10,2, 3,12, 5,22);
10    compare("cmpSum - BY SUM OF DIGITS", cmpSum,
11        10,2, 3,12, 5,22);
12    compare("cmpVaR - BY VALUE REVERSED", cmpVaR,
13        10,2, 3,12, 5,22);
14    }
15
16    private static void compare(String message,
17        MyCompar cmp, int... pairs) {
18        System.out.println("\n===== " + message);
19        for (int k = 0; k < pairs.length; k += 2) {
20            int a = pairs[k], b = pairs[k+1];
21            System.out.println("** (" + a + ", " + b + "): " +
22                "lt->" + cmp.lt(a,b) + ", " +
23                "le->" + cmp.le(a,b) + "\n" +
24                "gt->" + cmp.gt(a,b) + ", " +
25                "ge->" + cmp.ge(a,b) + ", " +
26                "eq->" + cmp.eq(a,b) + ", " +
27                "ne->" + cmp.ne(a,b) );
28        }
29    }
30 }

```

The program prints

```

===== cmpVal - BY VALUE
** (10,2): lt->>false, le->>false
    gt->true, ge->true, eq->>false, ne->true
** (3,12): lt->true, le->true
    gt->>false, ge->>false, eq->>false, ne->true
** (5,22): lt->true, le->true
    gt->>false, ge->>false, eq->>false, ne->true

===== cmpSum - BY SUM OF DIGITS
** (10,2): lt->true, le->true
    gt->>false, ge->>false, eq->>false, ne->true
** (3,12): lt->>false, le->true
    gt->>false, ge->true, eq->true, ne->>false
** (5,22): lt->>false, le->>false
    gt->true, ge->true, eq->>false, ne->true

===== cmpVaR - BY VALUE REVERSED
** (10,2): lt->true, le->true
    gt->>false, ge->>false, eq->>false, ne->true
** (3,12): lt->>false, le->>false
    gt->true, ge->true, eq->>false, ne->true
** (5,22): lt->>false, le->>false
    gt->true, ge->true, eq->>false, ne->true

```

Let us now consider an example of a functional interface with abstract method. In the program below, we define **Fun** interface which declares one abstract method

(**apply**) but is additionally equipped with a static function (**transformArray**) taking one array of doubles and returning another whose elements are the results of applying a function to elements of the input array. The function takes, as its second argument, the reference to an object of any class implementing the **Fun** interface and therefore providing a definition of the **apply** method:

Listing 28

ELL-InterFun/InterFun.java

```

1 public class InterFun {
2     public static void main(String[] args) {
3         double[] a = {-Math.PI/6, Math.PI/6, 7*Math.PI/6};
4         double[] r = Fun.transformArray(a, new Sin());
5         double[] s = Fun.transformArray(r, new MultBy2());
6         System.out.println(java.util.Arrays.toString(s));
7     }
8
9 }
10
11 @FunctionalInterface
12 interface Fun {
13     double apply(double d);
14
15     static double[] transformArray(double[] arr, Fun f) {
16         double[] res = new double[arr.length];
17         for (int i = 0; i < arr.length; ++i)
18             res[i] = f.apply(arr[i]);
19         return res;
20     }
21 }
22
23 class Sin implements Fun {
24     @Override
25     public double apply(double d) { return Math.sin(d); }
26 }
27
28 class MultBy2 implements Fun {
29     @Override
30     public double apply(double d) { return 2*d; }
31 }

```

Here, we first apply the **sin** function to an array of doubles, and then we apply multiplication by 2 to the resulting array. The program prints

```
[-0.9999999999999999, 0.9999999999999999, -0.9999999999999994]
```

Many important interfaces are already defined in the standard library. For example, **Comparable** declares one abstract method

```
int compareTo(Object)
```

Given objects *a* and *b*, *a.compareTo(b)* returns something negative if *a* is “smaller” than *b*, something positive if *b* is smaller, and 0 if they are considered equal. When

declaring that a class implements this interface for objects of type **T**, we should always indicate this type (in angle brackets) — then we can declare the type of the argument as **T** and not **Object** (and no casting is required). Classes implementing **Comparable** are said to be equipped with **natural order**.

In the (abstract) class **Figure** below, we declare and define **compareTo** (remember that all methods declared in an interface are by definition public, so overriding them we must not forget about **public**) specifically for **Figures** and therefore we declare the class as implementing **Comparable<Figure>**:

Listing 29

EQA-AbstractFigs/Figure.java

```

1  abstract class Figure implements Comparable<Figure> {
2
3      abstract public double getArea();
4      abstract public double getPerimeter();
5
6      static public Figure getFigMaxArea(Figure[] figs) {
7          double maxarea = 0;
8          Figure maxfig = null;
9          for (Figure f : figs) {
10             double area = f.getArea();
11             if (area > maxarea) {
12                 maxarea = area;
13                 maxfig = f;
14             }
15         }
16         return maxfig;
17     }
18
19     @Override
20     public String toString() {
21         return " area: "
22             + String.format("%6.3f",getArea())
23             + "; perimeter: "
24             + String.format("%6.3f",getPerimeter());
25     }
26
27     @Override
28     public int compareTo(Figure f) {
29         double diff = getPerimeter() - f.getPerimeter();
30         if (diff < 0) return -1;
31         else if (diff > 0) return +1;
32         else return 0;
33     }
34 }

```

Then we can define implementing classes: **Circle**

Listing 30

EQA-AbstractFigs/Circle.java

```
1 public class Circle extends Figure {
2     private double r;
3
4     public Circle(double r) {
5         this.r = r;
6     }
7
8     @Override
9     public double getArea() {
10         return Math.PI*r*r;
11     }
12     @Override
13     public double getPerimeter() {
14         return 2*Math.PI*r;
15     }
16     @Override
17     public String toString() {
18         return "Circle ( r=" + r + " )"
19             + super.toString();
20     }
21 }
```

and **Rectangle**

Listing 31

EQA-AbstractFigs/Rectangle.java

```
1 public class Rectangle extends Figure {
2     private double a, b;
3
4     public Rectangle(double a, double b) {
5         this.a = a;
6         this.b = b;
7     }
8
9     @Override
10    public double getArea() {
11        return a*b;
12    }
13    @Override
14    public double getPerimeter() {
15        return 2*(a+b);
16    }
17
18    @Override
19    public String toString() {
20        return "Rectangle (a=" + a + " b=" + b + ")"
21            + super.toString();
22    }
23 }
```

23 }

Then in **main** we can use **sort** and Java will know how to compare figures: it will just call **compareTo** (it knows it is there, because the compiler can see that **Figure** implements **Comparable**, otherwise the program would not even compile):

Listing 32

EQA-AbstractFigs/Main.java

```

1  import java.util.Arrays;
2  import java.util.Locale;
3
4  public class Main {
5      public static void main(String[] args) {
6          // to have decimal point instead of a comma...
7          Locale.setDefault(Locale.US);
8
9          Figure[] figs = {
10             new Circle(2),      new Rectangle(9,1),
11             new Rectangle(4,3), new Circle(4)
12         };
13
14         Figure fig = Figure.getFigMaxArea(figs);
15         System.out.println("\nLargest area: \n" + fig);
16
17         Arrays.sort(figs);
18         System.out.println("\nSorted by circumference:");
19         for (Figure f : figs)
20             System.out.println(f);
21     }
22 }
```

The program prints

Largest area:

Circle (r=4.0) area: 50.265; perimeter: 25.133

Sorted by circumference:

Circle (r=2.0) area: 12.566; perimeter: 12.566

Rectangle (a=4.0 b=3.0) area: 12.000; perimeter: 14.000

Rectangle (a=9.0 b=1.0) area: 9.000; perimeter: 20.000

Circle (r=4.0) area: 50.265; perimeter: 25.133

Any class can implement only one natural order. However it may happen that we want to use (e.g., for sorting) different criteria. We then can create an object which will be used as a comparator even if a natural order exist: it will be an object of a type implementing **Comparator** with only one abstract method

```
int compare(Object, Object)
```

Then, if **ob** is an object of this class and **a** and **b** are to be compared, **ob.compare(a,b)** should return something negative if **a** is “smaller” than **b**, something positive if **b** is smaller, and 0 if they are considered equal. As with **Comparable**, when declaring

a class implementing **Comparator**, we should always indicate the type, call it **T**, of objects it is supposed to be able to compare (in angle brackets) — then we can declare the type of the arguments of **compare** as **T** and not **Object** (and no casting is required).

Let us consider an example: class **Person** has a natural order (as it implements **Comparable**)

Listing 33

ELP-Comps2/Person.java

```

1 public class Person implements Comparable<Person> {
2
3     final static int currentYear =
4         java.util.Calendar.getInstance().
5             get(java.util.Calendar.YEAR);
6
7     String name;
8     int    birthYear;
9     int    height;
10
11     Person(String n, int y, int h) {
12         name      = n;
13         birthYear = y;
14         height    = h;
15     }
16
17     /**
18      * natural order: by name, then age, then height
19      */
20     @Override
21     public int compareTo(Person o) {
22         int k = name.compareToIgnoreCase(o.name);
23         if ( k != 0 ) return k;
24         k = o.birthYear - birthYear;
25         if ( k != 0 ) return k;
26         return height - o.height;
27     }
28
29     @Override
30     public String toString() {
31         return name + "(" + (currentYear - birthYear) +
32             "/" + height + ")";
33     }
34 }

```

In order to be able to compare persons in different ways, not necessarily determined by the natural order, we define two different classes representing comparators of **Persons**

Listing 34

ELP-Comps2/Comparators.java

```

1  import java.util.Comparator;
2
3  /**
4   * Comparator 1: by height, then age , then name
5   */
6  class Comp1 implements Comparator<Person> {
7      @Override
8      public int compare(Person o1, Person o2) {
9          int k = o1.height - o2.height;
10         if ( k != 0 ) return k;
11         k = o2.birthYear - o1.birthYear;
12         if ( k != 0 ) return k;
13         return o1.name.compareToIgnoreCase(o2.name);
14     }
15 }
16
17 /**
18  * Comparator 2: by age, then by name, then by height
19  */
20 class Comp2 implements Comparator<Person> {
21     @Override
22     public int compare(Person o1, Person o2) {
23         int k = o2.birthYear - o1.birthYear;
24         if ( k != 0 ) return k;
25         k = o1.name.compareToIgnoreCase(o2.name);
26         if ( k != 0 ) return k;
27         return o1.height-o2.height;
28     }
29 }

```

which now can be used, for example, to sort arrays or lists of **Persons**:

Listing 35

ELP-Comps2/Main.java

```

1  import java.util.Arrays;
2  import java.util.Comparator;
3
4  public class Main {
5      public static void main(String[] args) {
6          new Main();
7      }
8
9      Main() {
10         Person[] list = {
11             new Person("K",1980,165),
12             new Person("B",1986,171),
13             new Person("K",1980,168),
14             new Person("H",1980,171),

```

```

15         new Person("M",1980,171),
16         new Person("K",1980,169),
17         new Person("B",1979,171),
18         new Person("G",1975,171)
19     };
20
21     // natural
22     Arrays.sort(list);
23     writeL(list, "Natural: name, age, height");
24
25     // comparator Comp1
26     Arrays.sort(list, new Comp1());
27     writeL(list, "Comp1: height, age, name");
28
29     // comparator Comp2
30     Comparator<Person> comp2 = new Comp2();
31     Arrays.sort(list, comp2);
32     writeL(list, "Comp2: age, name, height");
33
34     // anonymous comparator
35     Arrays.sort(list, new Comparator<Person>() {
36         @Override
37         public int compare(Person p, Person q) {
38             int k = p.name.compareToIgnoreCase(q.name);
39             if ( k != 0 ) return k;
40             k = p.height - q.height;
41             if ( k != 0 ) return k;
42             return q.birthYear - p.birthYear;
43         }
44     });
45     writeL(list, "Anonym: name, height, age");
46
47     // lambda
48     Arrays.sort(list, (f,s) -> f.height-s.height);
49     writeL(list, "Lambda: name, height, age");
50 }
51
52 static void writeL(Person[] list, String header) {
53     System.out.println('\n'+header);
54     for (Person p : list) System.out.print(p+" ");
55     System.out.println();
56 }
57 }

```

The program prints

```

Natural: name, age, height
B(33/171) B(40/171) G(44/171) H(39/171)
K(39/165) K(39/168) K(39/169) M(39/171)

```

Comp1: height, age, name
 K(39/165) K(39/168) K(39/169) B(33/171)
 H(39/171) M(39/171) B(40/171) G(44/171)

Comp2: age, name, height
 B(33/171) H(39/171) K(39/165) K(39/168)
 K(39/169) M(39/171) B(40/171) G(44/171)

Anonym: name, height, age
 B(33/171) B(40/171) G(44/171) H(39/171)
 K(39/165) K(39/168) K(39/169) M(39/171)

Lambda: name, height, age
 K(39/165) K(39/168) K(39/169) B(33/171)
 B(40/171) G(44/171) H(39/171) M(39/171)

Let us consider another, but similar, example. We again create a class representing persons (equipped with a natural order)

Listing 36

ELM-Comps1/Person.java

```

1 public class Person implements Comparable<Person> {
2
3     private String name;
4     private int    year;
5
6     public Person(String name, int year) {
7         this.name = name;
8         this.year = year;
9     }
10
11     @Override
12     public int compareTo(Person other) {
13         int diff = year - other.year;
14         if (diff != 0) return diff;
15         else          return name.compareTo(other.name);
16     }
17
18     public String getName() { return name; }
19     public int  getYear()   { return year; }
20
21     @Override
22     public String toString() {
23         return name + "(" + year + ")";
24     }
25
26     static void show(Person[] persons, String message) {
27         System.out.println(message);
28         for (Person person : persons)
29             System.out.print(person + " ");

```

```

30         System.out.println("\n");
31     }
32 }

```

Now, we create only one class, **CompPerson**, objects of which can be used as comparators of **Persons**. The class contains one field, set by the constructor. In the example below it is an enumerator, but equally well it could have been an integer. When creating an object of this class, we will pass to the constructor information about the way we want persons to be compared — therefore, our class is in a way ‘configurable’

Listing 37

ELM-Comps1/CompPerson.java

```

1  import java.util.Comparator;
2
3  public class CompPerson implements Comparator<Person> {
4
5      public static enum Comp { BY_NAME,    BY_YEAR,
6                               BY_NAMERev, BY_YEARRev };
7
8      private Comp comp;
9
10     public CompPerson(Comp comp) {
11         this.comp = comp;
12     }
13
14     @Override
15     public int compare(Person p1, Person p2) {
16
17         int rYear = p1.getYear() - p2.getYear();
18         int rName = p1.getName().compareTo(p2.getName());
19
20         return switch (comp) {
21             case BY_NAME      -> rName != 0 ?  rName : rYear;
22             case BY_NAMERev   -> rName != 0 ? -rName : rYear;
23             case BY_YEAR      -> rYear != 0 ?  rYear : rName;
24             case BY_YEARRev   -> rYear != 0 ? -rYear : rName;
25         };
26     }
27 }

```

Then we can use objects of this class as a comparator to sort collections (or arrays) of **Persons** in various ways:

Listing 38

ELM-Comps1/Main.java

```

1  import java.util.Arrays;
2
3  public class Main {
4

```



```

5   public static void main(String[] args) {
6       Person[] persons = {
7           new Person("Mary",1990),
8           new Person("Joan",1992),
9           new Person("Suzy",1992),
10          new Person("Beth",1992),
11          new Person("Suzy",1980),
12          new Person("Katy",1982),
13      };
14      Person.show(persons,"At the beginning:");
15
16      Arrays.sort(persons);
17      Person.show(persons,"Natural order: " +
18                  "by year, then by name");
19
20      Arrays.sort(persons,
21                  new CompPerson(CompPerson.Comp.BY_NAME));
22      Person.show(persons,"Order BY_NAME: " +
23                  "by name, then by year");
24
25      Arrays.sort(persons,
26                  new CompPerson(CompPerson.Comp.BY_NAMERev));
27      Person.show(persons,"Order BY_NAMERev: " +
28                  "by name reversed, then by year");
29
30      Arrays.sort(persons,
31                  new CompPerson(CompPerson.Comp.BY_YEAR));
32      Person.show(persons,"Order BY_YEAR: " +
33                  "by year, then by name");
34
35      Arrays.sort(persons,
36                  new CompPerson(CompPerson.Comp.BY_YEARRev));
37      Person.show(persons,"Order BY_YEARRev: " +
38                  "by year reversed, then by name");
39
40      Arrays.sort(persons,
41                  (f,s) -> s.getYear() - f.getYear());
42      Person.show(persons,"Order by lambda : " +
43                  "by year ");
44  }
45  }

```

The program prints

At the beginning:

Mary(1990) Joan(1992) Suzy(1992) Beth(1992) Suzy(1980) Katy(1982)

Natural order: by year, then by name

Suzy(1980) Katy(1982) Mary(1990) Beth(1992) Joan(1992) Suzy(1992)


```

15     }
16 }
17
18 public Inner getInner(String i) {
19     Inner inn = new Inner(i);
20     System.out.println("Creating inner " + inn.sInn);
21     return inn;
22 }
23
24 @Override
25 public String toString() { return "Outer-" + sOut; }
26 }
27
28 public class OutInn {
29     public static void main (String[] args) {
30         Outer out1 = new Outer("out1");
31         Outer.Inner inn1 = out1.getInner("inn1");
32         Outer.Inner inn2 = out1.new Inner("inn2");
33         System.out.println(out1);
34         System.out.println(inn1);
35         System.out.println(inn2);
36         System.out.println(out1.getClass().getName());
37         System.out.println(inn1.getClass().getName());
38         System.out.println(inn2.getClass().getName());
39     }
40 }

```

which prints

```

Creating inner inn1
Outer-out1
Inner-inn1 parent Outer-out1
Inner-inn2 parent Outer-out1
Outer
Outer$Inner
Outer$Inner

```

3.3.2 Static inner classes

An inner class may also be declared as **static**. It is still a “friend” of the outer class, but there is no `Outer.this` inside the object of the inner class; therefore, objects of the inner class may exist independently of any objects of the outer class.

In the example below class **MyStack** represents a stack (of integers) implemented as a singly linked list. We need a class representing individual nodes of the list, but the user of the stack doesn’t need to know about its existence; therefore we define **Node** as a private static inner class inside **MyStack**:

Listing 40

GMC-StackSimple/MyStack.java

```
1 public class MyStack {
2     // static inner class
3     private static class Node {
4         int data;
5         Node next;
6         Node(int d, Node n) {
7             data = d;
8             next = n;
9         }
10    }
11
12    private Node top;
13
14    public MyStack() {
15        top = null;
16    }
17    public void push(int d) {
18        top = new Node(d, top);
19    }
20    public int pop() {
21        int d = top.data;
22        top = top.next;
23        return d;
24    }
25    public boolean empty() {
26        return top == null;
27    }
28 }
```

and the user uses only objects of type **MyStack**; class **Node** is just an “implementation detail”:

Listing 41

GMC-StackSimple/StackSimple.java

```
1 public class StackSimple {
2     public static void main (String[] args) {
3         MyStack stInt = new MyStack();
4         for (int i = 5; i > 0; --i)
5             stInt.push(i);
6         while (!stInt.empty())
7             System.out.print(stInt.pop() + " ");
8         System.out.println();
9     }
10 }
```

3.3.3 Anonymous classes

Sometimes we have to create just one object of a type which implements an interface, or extends an abstract class, or behaves as an object of an existing concrete class but with one or a few methods overridden: in such situation one can use object of an **anonymous class**. The syntax is illustrated in the example below: after **new** we specify a class (concrete or abstract) that we want our anonymous class to extend, or an interface that we want it to implement. In the first case we can also pass arguments to a constructor; in any case round parentheses are obligatory. Then, in curly braces, we write an implementation (normally, we just override one or more methods). The compiler will then create an anonymous class and return an object of this type:

Listing 42

ELI-Anon/Anon.java

```

1  interface BiIntOperator {
2      int apply(int i, int j);
3  }
4
5  class AddAndMult implements BiIntOperator {
6      int seed;
7      AddAndMult(int seed) { this.seed = seed; }
8      AddAndMult()          { this(1);          }
9      @Override
10     public int apply(int i, int j) { return seed*(i + j); }
11 }
12
13 class MultAndAdd implements BiIntOperator {
14     int seed;
15     MultAndAdd(int seed) { this.seed = seed; }
16     MultAndAdd()          { this(1);          }
17     @Override
18     public int apply(int i, int j) { return seed + i*j; }
19 }
20
21 public class Anon {
22     public static void main(String[] args) {
23         BiIntOperator[]opers = {
24             // objects of concrete classes
25             // implementing an interface
26             new AddAndMult(2),
27             new MultAndAdd(5),
28             // object of anonymous class
29             // implementing an interface
30             new BiIntOperator() {
31                 @Override
32                 public int apply(int i, int j) {
33                     return i*i + j*j;
34                 }
35             },
36             // object of anonymous class
37             // extending a 'normal' class

```

```

38         new MultAndAdd(3) {
39             @Override
40             public int apply(int i, int j) {
41                 return seed*(i*i + j*j);
42             }
43         }
44     };
45     int a = 1, b = 2;
46     for (BiIntOperator op : opers)
47         System.out.print(op.apply(a,b) + " ");
48     System.out.println();
49 }
50 }

```

(the program prints 6 7 5 15).

Of course, we cannot create another object of such an anonymous class, because it doesn't even have a name. For the same reason, anonymous classes cannot define any constructors.

Another example illustrating abstract and anonymous classes: we define an abstract class **Animal**

Listing 43

EQE-AbstractAnimals/Animal.java

```

1 public abstract class Animal {
2     String name;
3     double weight;
4
5     public Animal(String name, double weight) {
6         this.name = name;
7         this.weight = weight;
8     }
9
10    abstract public String speak();
11
12    @Override
13    public String toString() {
14        return name + "(" + weight + ") - " + speak();
15    }
16 }

```

and then we use it in a program creating various animals

Listing 44

EQE-AbstractAnimals/Main.java

```

1 public class Main {
2     public static void main(String[] args) {
3
4         Animal max =

```

```

5         new Animal("Max", 15) {
6             @Override
7             public String speak() {
8                 return "bow-wow";
9             }
10        };
11
12        Animal[] animals =
13        {
14            max,
15            new Animal("Batty", 3.5) {
16                @Override
17                public String speak() {
18                    return "miaou-miaou";
19                }
20            }
21        };
22
23        for (Animal a : animals)
24            System.out.println(a);
25    }
26 }

```

Program prints

```

Max(15.0) - bow-wow
Batty(3.5) - miaou-miaou

```

Important: When we define an anonymous class, local variables visible in the current scope will be accessible (and can be used) inside the body of methods of the anonymous class being created. The only condition is that these local variables are

- declared as **final** (and, of course, initialized), or
- **effectively final**, i.e., they are defined and initialized and then not modified.

In the program below, for example, the variable `meow` is defined in the **main** function as a local variable (line 3).

Listing 45

EQH-Closure/Closure.java

```

1 public class Closure {
2     public static void main(String[] args) {
3         String meow = "Meow-meow";
4         Dog dog = new Dog("Kitty") {
5             @Override
6             public void voice() {
7                 System.out.println(name + ": " + meow);
8             }
9         };
10        dog.voice(); // prints 'Kitty: Meow-meow'

```

```

11     }
12 }
13
14 class Dog {
15     protected String name;
16     public Dog(String n) { name = n; }
17     public void voice() {
18         System.out.println(name + ": " + "bow-wow");
19     }
20 }

```

However, we can use it inside an anonymous class (line 7), as it is “effectively final”: it is defined and initialised, but never modified.

3.4 Lambdas

Instead of passing an object of a concrete or anonymous class implementing an interface, one can often provide just a simple expression describing a functionality of a method which is supposed to be implemented (we call such an expression a **lambda**). For this to be possible, the compiler has to know which method it is and from which interface. Therefore, we can use a lambda only when it is clear from the context implementation of which interface is expected. In order for the compiler to know what method is to be overridden, there must be only one abstract method in the interface involved — such interfaces, with only one abstract method, are called **functional interfaces**.

A lambda expression itself is composed of three parts:

- List of parameters, as in declaration of a ‘normal’ function. However, usually the types of parameters need not be specified, because the compiler is able to deduce them from the context. If there is only one parameter and type is omitted, parentheses are optional. If the list of parameters is empty, we just write empty parentheses.
- The “arrow” token: `->`.
- A function body enclosed in curly braces. If the body contains just one expression (i.e., something that has a value), there are no braces, no semicolon at the end and no **return** statement: the value of the expression will be evaluated and returned — return type will be deduced as the type of this value. Braces may be also omitted if the body consists of one statement which does not have any value — return type **void** will then be assumed.

Examples:

```

(int x, int y) -> x + y
(x, y) -> x*y < 0
() -> Math.random()
e -> System.out.println(e)

```

In these examples we assume that the compiler will be able to infer all necessary types and deduce what functional interface is to be implemented.

The example below illustrates both cases: when the body of a lambda is a single expression (no semicolon, no **return**, no braces) and when it is implemented as a compound statement (with braces, **return** and semicolons after statements, as usually). Note that

the compiler expects, as the second argument of the **sort** function ‘something that implements the **Comparator** interface’. As what is to be sorted is an array of **Persons**, the compiler knows also that it should be in fact **Comparator<Person>**, so the type of p1 and p2 is deduced to be **Person**:

Listing 46

EMD-InterF/InterF.java

```

1  import java.util.Arrays;
2
3  class Person {
4
5      private String name;
6      private int    year;
7
8      public Person(String name, int year) {
9          this.name = name;
10         this.year = year;
11     }
12
13     public String getName() { return name; }
14     public int   getYear()   { return year; }
15
16     @Override
17     public String toString() {
18         return name + "(" + year + ")";
19     }
20
21     static void show(Person[] persons, String message) {
22         System.out.println(message);
23         for (Person person : persons)
24             System.out.print(person + " ");
25         System.out.println();
26     }
27 }
28
29 public class InterF {
30
31     public static void main(String[] args) {
32         Person[] persons =
33             { new Person("Mary",1990),
34               new Person("Joan",1992),
35               new Person("Suzy",1992),
36               new Person("Beth",1992),
37               new Person("Suzy",1980),
38               new Person("Katy",1982), };
39         Person.show(persons,"At the beginning:");
40
41         // lambda as a single expression -
42         // no return, no semicolon
43         Arrays.sort(persons,

```

```

44         (p1, p2) -> p1.getYear()-p2.getYear());
45     Person.show(persons, "Ordered by age");
46
47     // lambda as a compound statement -
48     // return and semicolons, as usually
49     Arrays.sort(persons, (p1, p2) ->
50     {
51         int d = p1.getName().compareTo(p2.getName());
52         if (d != 0) return d;
53         return p1.getYear() - p2.getYear();
54     });
55     Person.show(persons, "Ordered by name then age");
56 }
57 }

```

The program prints

At the beginning:

Mary(1990) Joan(1992) Suzy(1992) Beth(1992) Suzy(1980) Katy(1982)

Ordered by age

Suzy(1980) Katy(1982) Mary(1990) Joan(1992) Suzy(1992) Beth(1992)

Ordered by name then age

Beth(1992) Joan(1992) Katy(1982) Mary(1990) Suzy(1980) Suzy(1992)

The scoping rules for lambdas are somewhat special. We can imagine that the compiler creates an object of an anonymous class implementing an interface and then implements its abstract method based on our lambda expression. However, it is not so — the body of a lambda behaves as a block inside the function it is defined in. This fact has several consequences, among others these

- inside the body of a lambda, one cannot define variables with same name as local variables from the surrounding scope;
- the reference **this** used inside the body of a lambda refers to an object of the surrounding class, *not* to an object of an anonymous class.

3.5 More examples

Let's look at some examples.

Listing 47

ELU-FInter/FInter.java

```

1  @FunctionalInterface
2  interface Calc {
3      boolean test(Double d);
4  }
5
6  @FunctionalInterface
7  interface Cons {
8      void consume(Object ob);
9  }

```

```

10
11 public class FInter {
12     public static void main (String[] args) {
13         double mn = 0, mx = 10; // effectively final
14         Calc[] arr = {
15             d -> d < 0,           // type of d inferred
16             d -> d >= 0,
17             d -> mn <= d && d <= mx
18         };
19
20         Cons cons = ob -> System.out.print(ob + " ");
21
22         for (double d = -2; d < 15; d += 5) {
23             for (Calc calc : arr)
24                 cons.consume(calc.test(d));
25             System.out.println();
26         }
27     }
28 }

```

The program prints

```

true false false
false true true
false true true
false true false

```

And another simple example. Here, we define an interface which declares a method **transform** which ‘transforms’ a strings — it takes a string and returns another string. Note that the function **transArray** in class **SimpleInter** takes, as its second argument ‘something that implements **Transformation** interface’. In **main** we call this function passing this ‘something’ in three different ways:

- as an object of a separate class **Reverse** implementing **Transformation**;
- as an object of an anonymous class implementing the same interface;
- as a lambda.

Listing 48

ELK-SimpleInter/SimpleInter.java

```

1 import java.util.Arrays;
2
3 interface Transformation {
4     String transform(String arg);
5 }
6
7 class Reverse implements Transformation {
8     @Override
9     public String transform(String s) {
10         char[] a = s.toCharArray();
11         for (int i = 0, j = s.length()-1; i < j; ++i, --j) {

```

```

12         char c = a[i];
13         a[i] = a[j];
14         a[j] = c;
15     }
16     return new String(a);
17 }
18 }
19
20 public class SimpleInter {
21     private static void transArray(String[] array,
22                                     Transformation t) {
23         for (int i = 0; i < array.length; ++i)
24             array[i] = t.transform(array[i]);
25     }
26
27     public static void main (String[] args) {
28         String[] arr = {"Mary", "Alice", "Janet", "Rachel"};
29         System.out.println(Arrays.toString(arr));
30
31         //object of a class
32         transArray(arr, new Reverse());
33         System.out.println(Arrays.toString(arr));
34
35         // object of an anonymous class
36         transArray(arr,
37                     new Transformation() {
38                         @Override
39                         public String transform(String s) {
40                             return s.toUpperCase();
41                         }
42                     });
43         System.out.println(Arrays.toString(arr));
44
45         // lambda
46         transArray(arr, s -> "" + s.charAt(0));
47         System.out.println(Arrays.toString(arr));
48     }
49 }

```

The program prints

```

[Mary, Alice, Janet, Rachel]
[yraM, ecilA, tenaJ, lehcaR]
[YRAM, ECILA, TENAJ, LEHCAR]
[Y, E, T, L]

```

Another important example of using a lambda has been already shown in one of the previous examples — see Listing 38. The **sort** function called with two arguments expects as the second one ‘something implementing the **Comparator** interface’. This

interface *is* a functional interface, because it declares only one abstract method: **compare**. Therefore, as the implementation of this method is usually rather short, it is very convenient to use lambdas instead of creating separate classes whose only purpose is to 'wrap' the **compare** method.

Section 4

Introduction to generic classes

This section covers a few basic concepts related to generics, or parametrized classes, in Java.¹

The subject is not easy; generics were added to Java rather late and their implementation is quite complicated, as it had to be consistent with earlier versions of the language. Let us also mention here that parametrized classes are somewhat related to class templates in C++ and Ada, but this similarity is rather misleading and superficial; it definitely should not be taken too literally.

4.1 Type parameters

Parametrized class uses as names of types arbitrary identifiers — called **type parameters** — which are then concretized, when we use objects of the generic type. Then we have to tell the compiler, explicitly or implicitly, what concrete class this identifier should denote. We introduce type parameters of a class like this:

```
class Pair<F,S> {
    // here we use F and S as names of types
}
```

Here **F** and **S** stand for names of some, as yet unknown, types. In the definition of the class we can use names **F** and **S** as names of classes (but *not* primitive types!). Of course, there are no classes **F** or **S**, so when creating an object of our class, we have to specify (in angle brackets) what these names should stand for:

```
Pair<Integer,String> p =
    new Pair<Integer,String>(2,"Sue");
    // or just (using diamond operator)
Pair<Integer,String> q = new Pair<>(1,"Lea");
```

In the second case, we used “type inferring” (diamond operator); we don’t need to repeat types on the right-hand side, because the compiler already knows them looking at the left-hand side (but angle brackets, although empty, are still required).

However, in the code produced by the compiler, type parameter will *not* be present: at run time types denoted by, say, **T** will be just **Object**. Information about the type of **T** is only used at compile time.

Let us consider, as an example, the following program:

Listing 49

GMA-GenerPair/Pair.java

```
1 import java.lang.reflect.Method;
2
3 public class Pair<F, S> {
4     private static int count = 0;
5     private F first;
6     private S second;
7     public Pair(F f, S s) {
```

¹By *class* we understand a “regular” class, an interface, an abstract class or a record (see Sec. ??, p. ??).

```

8      count++;
9      first  = f;
10     second = s;
11 }
12 public F getFirst() {return first; }
13 public S getSecond() {return second;}
14 public void setFirst(F f) {first  = f;}
15 public void setSecond(S s) {second = s;}
16 public String toString() {return first + " " + second;}
17
18 public static void main(String[] args) throws Exception {
19     Pair<String, Integer> p1 = new Pair<>("A",1);
20     Pair<String, String>  p2 = new Pair<>("C", "D");
21
22     // what are the dynamic types of p1 and p2 ?
23     System.out.println("Class of p1: " + p1.getClass() +
24         " ; Class od p2: " + p2.getClass());
25
26     // method signatures
27     for (Method m : p1.getClass().getDeclaredMethods())
28         if (!m.getName().equals("main"))
29             System.out.println(m); // type erasure
30
31     // only one static member 'count'
32     System.out.println(p1.count + " " + p2.count);
33
34     // casting not needed, autoboxing int -> Integer
35     p1.setSecond(2);
36
37     // automatic unboxing Integer -> int
38     int i = p1.getSecond();
39
40     // no casting, must be a String
41     String s = p2.getFirst();
42
43     System.out.println("p1.second = " + i);
44     System.out.println("p2.first  = " + s);
45 }
46 }

```

As we can see, the compiler remembers types used when creating objects, so casting is not needed; calling **getFirst** on **p2** must give us a **String** and invoking **setSecond** on **p1** we pass an **int** and the compiler knows that it has to be converted to **Integer**. However, the dynamic type, or, strictly speaking, the so called “raw” type of both **p1** and **p2** is just **Pair**. We can see it from the output:

```

Class of p1: class Pair; Class od p2: class Pair
public java.lang.Object Pair.getSecond()
public void Pair.setFirst(java.lang.Object)
public void Pair.setSecond(java.lang.Object)

```

```

public java.lang.String Pair.toString()
public java.lang.Object Pair.getFirst()
2 2
p1.second = 2
p2.first  = C

```

Therefore, at run time, there exist only one type, namely just **Pair**. Notice that at run time arguments of setters (**setFirst** and **setSecond**) and return type of getters (**getFirst** and **getSecond**) are of type **Object**: this is known as **type erasure**. Only at compile time the compiler knows the required (declared) types of the two elements of a pair and will not allow us to insert (for example using **setFirst**) an object of a wrong type. Also notice that there exist only one static member **count** because dynamic types of both **p1** and **p2** are the same — just **Pair**.

Parametrized type can be, and are, useful, because they

- allow us to avoid explicit conversions, because the compiler knows expected types of arguments and return values of methods;
- make the code shorter and simpler to read and understand;
- allow the compiler to detect many errors related to type mismatch, which would otherwise manifest themselves at run time, triggering, for example, **ClassTypeExceptions**.

On the other hand, mainly because of the type erasure that we have just mentioned, there are some quite severe restrictions when defining and using parametrized types. If **T** is a type parameter

- We cannot create objects of type **T**;
- We cannot create arrays of (references to) objects of type **T** (but it *is* possible to create collections parametrized by **T**);
- We cannot use **instanceof** operator with **T**;
- Type **T** cannot be used for static fields (as parametrized class corresponds to one raw type);

Let us look at another example of a parametrized class: a class representing a queue. Making our class generic (parametrized) allows us to create queues of elements of various types preserving strict type checking:

Listing 50

GME-QueueGener/MyQueue.java

```

1 public class MyQueue<E> {
2
3     private class Node {
4         E    data;
5         Node next = null;
6         Node(E d) { data = d; }
7     }
8     private Node head, tail;
9
10    public MyQueue() {
11        head = tail = null;
12    }
13    public void enqueue(E d) {

```



```

14     if (head == null)
15         head = tail = new Node(d);
16     else
17         tail = tail.next = new Node(d);
18 }
19 public E dequeue() {
20     E d = head.data;
21     if ((head = head.next) == null) tail = null;
22     return d;
23 }
24 public boolean empty() {
25     return head == null;
26 }
27 }

```

In **main** we create queues of **Strings** and **Doubles** and use them in a uniform way:

Listing 51

GME-QueueGener/QueueGener.java

```

1 public class QueueGener {
2     public static void main (String[] args) {
3         MyQueue<String> queueS = new MyQueue<>();
4         MyQueue<Double> queueD = new MyQueue<>();
5         for (double d = 0.5; d < 5; d += 1) {
6             queueS.enqueue(String.valueOf(d));
7             queueD.enqueue(d); // boxing
8         }
9         while (!queueS.empty() && !queueD.empty()) {
10             // no casting required
11             String s = queueS.dequeue();
12             double d = queueD.dequeue();
13             System.out.println(
14                 "String: " + s + " " +
15                 "Double: " + d);
16         }
17     }
18 }

```

In the program, class **MyQueue** implements a queue of objects of any type. This does not mean that we can enqueue any object to any queue: when creating a queue, we decide what the type of enqueued objects should be. For example, the queue **queueS** is declared as queue of **Strings**. Therefore, the compiler will not allow us to enqueue anything other than **String** (and in the case of **queueD** — **Doubles**). Also notice that when enqueueing and dequeueing elements, we don't have to use casting: the compiler knows what type is expected and will even automatically perform boxing/unboxing of primitive types for us.

Let us consider another example. We define a generic interface **Operat** which declares one function representing an operator, i.e. a function taking two arguments of the same type and returning a result also of this type. Object implementing this

interface will then be used by static functions defined in class **Tools**. Note, how we define generic *static functions* — here we inform the compiler that **T** is not a class but a type parameter for every static function defined in the class separately; one of these functions performs the so called *left-folding* and the other combines two sequences into one. We don't create any concrete classes implementing **Operat**; instead, in **main**, we just pass objects of anonymous classes implementing it:

Listing 52

ELZ-IFac/IFaces.java

```

1  import java.util.ArrayList;
2  import java.util.Arrays;
3  import java.util.List;
4
5  interface Operat<T> {
6      T oper(T lhs, T rhs);
7  }
8
9  class Tools {
10     public static <T> T foldl(
11         Operat<T> op, List<T> list, T id) {
12         T acc = id;
13         for (T e : list)
14             acc = op.oper(acc,e);
15         return acc;
16     }
17
18     public static <T> List<T> combine(
19         Operat<T> op, List<T> l1, List<T> l2) {
20         List<T> res = new ArrayList<T>();
21         int size = Math.min(l1.size(), l2.size());
22         for (int i = 0; i < size; ++i)
23             res.add(op.oper(l1.get(i), l2.get(i)));
24         return res;
25     }
26 }
27
28 public class IFaces {
29     public static void main (String[] args) {
30         Operat<Integer> operInt = new Operat<Integer>() {
31             @Override
32             public Integer oper(Integer lhs, Integer rhs) {
33                 return lhs + rhs;
34             }
35         };
36         Operat<String> operStr = new Operat<String>() {
37             @Override
38             public String oper(String lhs, String rhs) {
39                 return lhs + rhs;
40             }
41         };

```

```

42
43     List<Integer> listInt1 = Arrays.asList(1,2,3,4),
44           listInt2 = Arrays.asList(5,6,7,8);
45     List<Integer> intRes =
46         Tools.combine(operInt, listInt1, listInt2);
47     System.out.println("intRes = " + intRes);
48
49     List<String> listStr1 = Arrays.asList("a","b","c"),
50           listStr2 = Arrays.asList("1","2","3");
51     List<String> strRes =
52         Tools.combine(operStr, listStr1, listStr2);
53     System.out.println("strRes = " + strRes);
54
55     int intFold = Tools.foldl(operInt, intRes, 0);
56     System.out.println("intFold = " + intFold);
57
58     String strFold = Tools.foldl(operStr, strRes, "");
59     System.out.println("strFold = " + strFold);
60 }
61 }

```

The program prints

```

intRes = [6, 8, 10, 12]
strRes = [a1, b2, c3]
intFold = 36
strFold = a1b2c3

```

Other examples: here we use generic functional interfaces.

The first example: here we define a generic functional interface which represents an *operator*, i.e., function accepting two objects of the same type and returning a result also of this type:

Listing 53

ELW-LambdaInter/MyBiOp.java

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  @FunctionalInterface
5  interface MyBiOpInterface<T> {
6      T apply(T a, T b);
7  }
8
9  class Mult implements MyBiOpInterface<Double> {
10     @Override
11     public Double apply(Double a, Double b) { return a*b; }
12 }
13
14 public class MyBiOp {

```

```

15     public static void main (String[] args) {
16         List<MyBiOpInterface<Double>>
17            opers = new ArrayList<>();
18
19         // addition - reference to (static) method
20         opers.add( Double::sum );
21
22         // subtraction - anonymous class
23         opers.add( new MyBiOpInterface<Double>() {
24             @Override
25             public Double apply(Double a, Double b) {
26                 return a - b;
27             }
28         });
29
30         // multiplication - object of a named class
31         // implementing the MyBiOpInterface interface
32         opers.add( new Mult() );
33
34         // division - lambda
35         opers.add( (a,b) -> a/b );
36
37         // with closure ('shift' is effectively final)
38         int shift = 10;
39         opers.add( (a,b) -> a/b + shift);
40
41         for (MyBiOpInterface<Double> op : opers)
42             System.out.println(op.apply(10.5,3.5));
43     }
44 }

```

The program prints

```

14.0
7.0
36.75
3.0
13.0

```

and another one

Listing 54

ELX-FuncInter/FuncInter.java

```

1  @FunctionalInterface
2  interface MyInterface<T> {
3      int len(T t);
4  }
5
6  public class FuncInter {
7      static <T> int calc(T arg, MyInterface<T> f) {

```

```

8         return f.len(arg);
9     }
10
11     public static void main (String[] args) {
12         String s = "Alice";
13         int result1 = calc(
14             s,
15             new MyInterface<String>() {
16                 @Override
17                 public int len(String s) {
18                     return s.length();
19                 }
20             });
21         System.out.println("result1 = " + result1);
22
23         Double d = 123.456; // boxing
24         int result2 =
25             calc(d, v -> v.toString().length());
26         System.out.println("result2 = " + result2);
27
28         int result3 =
29             calc(d, v -> (int)(1000*v+4));
30         System.out.println("result3 = " + result3);
31     }
32 }

```

The program prints

```

result1 = 5
result2 = 7
result3 = 123460

```

The following example is very similar to that in Listing 52, but now we use lambdas: directly as an argument, or to create a variable which could be used more than once:

Listing 55

EMA-IFacLam/IFacesLam.java

```

1 import java.util.ArrayList;
2 import java.util.Arrays;
3 import java.util.List;
4
5 interface Operat<T> {
6     T oper(T lhs, T rhs);
7 }
8
9 class Tools {
10     public static <T> T foldl(
11         Operat<T> op, List<T> list, T id) {
12         T acc = id;
13         for (T e : list)

```

```

14         acc = op.oper(acc,e);
15     return acc;
16 }
17
18     public static <T> List<T> combine(
19         Operat<T> op, List<T> l1, List<T> l2) {
20         List<T> res = new ArrayList<T>();
21         int size = Math.min(l1.size(), l2.size());
22         for (int i = 0; i < size; ++i)
23             res.add(op.oper(l1.get(i), l2.get(i)));
24         return res;
25     }
26 }
27
28 public class IFacesLam {
29     public static void main (String[] args) {
30         Operat<String> operStr = (lhs, rhs) -> lhs + rhs;
31
32         List<Integer> listInt1 = Arrays.asList(1,2,3,4),
33             listInt2 = Arrays.asList(5,6,7,8);
34         List<Integer> intRes =
35             Tools.combine((lhs, rhs) -> lhs + rhs,
36                 listInt1, listInt2);
37         System.out.println("intRes = " + intRes);
38
39         List<String> listStr1 = Arrays.asList("a","b","c"),
40             listStr2 = Arrays.asList("1","2","3");
41         List<String> strRes =
42             Tools.combine(operStr, listStr1, listStr2);
43         System.out.println("strRes = " + strRes);
44
45         int intFold = Tools.foldl(
46             (lhs, rhs) -> lhs + rhs, intRes, 0);
47         System.out.println("intFold = " + intFold);
48
49         String strFold = Tools.foldl(operStr, strRes, "");
50         System.out.println("strFold = " + strFold);
51     }
52 }

```

The program prints

```

intRes = [6, 8, 10, 12]
strRes = [a1, b2, c3]
intFold = 36
strFold = a1b2c3

```

The last example illustrates an interface with one *defined* default method; note, that we have to inform the compiler that **S** is an additional type parameter (**T** and **R** have already been declared as such.) The interface represents an operation (mapping),

called here **apply**, from values of one type to values of another type ($T \rightarrow R$). However, the interface *defines* also a method (**compos**). This is a method, so it is called on an object representing one operation (say, $f : T \rightarrow R$), takes a value **arg** of type **T** as an argument and an object representing another operation (say, $g : R \rightarrow S$). It then returns the result of the composition $(g \circ f)(arg)$. The following program

Listing 56

EMB-Compos/Composit.java

```

1  @FunctionalInterface
2  interface Func<T,R> {
3      R apply(T e);
4      default <S> S compos(T arg, Func<R,S> g) {
5          return g.apply(apply(arg));
6      }
7  }
8
9  public class Composit {
10     public static void main(String[] args) {
11         Func<String,Integer> f = s -> s.length();
12         System.out.println("g(f(\"abc\")) = " +
13             f.compos("abc", v -> v*Math.PI));
14     }
15 }

```

prints

$g(f("abc")) = 9.42477796076938$

as the length of "abc" is 3 and $3\pi \approx 9.42477796076938$. Note that the **g** operation was given as a lambda but still the compiler was able to deduce that the type **S** of the result should be **Double**.

4.2 Bound types

Specifying a type parameter, say **T**, we can limit possible types that it can represent to types meeting some requirements. This allows us, for example, to use, on objects of **T**, methods not necessarily only from class **Object** but more specific methods from other classes that has been specified as type bounds. Such restrictions define sets of classes that can be substituted for **T**: then we can use methods which the classes from these sets contain. The syntax looks like this:

T extends Type1 & Type2 & Type3 & ... & TypeN

where:

- **T** is the type parameter;
- **Type1** is the name of a class or an interface;
- **Type2** ... **TypeN** are names of interfaces.

As we can see, if among restrictions there is a (concrete or abstract) class, it must be specified as the first in the list, all others bounds must be interfaces. Types **Type1** ... **TypeN** may be themselves parametrized (generic) classes or interfaces.

Objects of types restricted in this way can be used as objects of type **Type1**: compilation will fail if we try to substitute for **T** something that does not extend or implement

Type1 or does not implement **Type2** ... **TypeN**. Therefore, it will be safe to call methods of all these restricting classes or interfaces.

In the example below, class **GenArr** is parametrized by type **T**, but not arbitrary but only by classes implementing **Comparable** — therefore, we can use the method **compareTo** on objects from the array (in order to find the minimum and maximum elements of the array):

Listing 57

GMH-MinMaxGener/MinMaxGener.java

```

1  class GenArr<T extends Comparable<T>> {
2      private T[] arr;
3      private T    min, max;
4
5      public GenArr(T[] arr) {
6          if (arr == null || arr.length == 0)
7              throw new IllegalArgumentException();
8          min = arr[0];
9          max = arr[0];
10         for (int i = 1; i < arr.length; ++i) {
11             // we can use compareTo, as we know
12             // that T extends Comparable
13             // (and the compiler has checked it)
14             if (arr[i].compareTo(min) < 0) min = arr[i];
15             if (arr[i].compareTo(max) > 0) max = arr[i];
16         }
17     }
18     public T getMin() { return min; }
19     public T getMax() { return max; }
20 }
21
22 public class MinMaxGener {
23     public static void main(String[] args) {
24         GenArr<Integer> mnmxI =
25             new GenArr<>(new Integer[]{3, -2, -7, 2});
26         GenArr<String> mnmxS =
27             new GenArr<>(new String[]{"A", "Z", "C"});
28
29         System.out.println("I - min = " + mnmxI.getMin() +
30                             "\nI - max = " + mnmxI.getMax());
31         System.out.println("S - min = " + mnmxS.getMin() +
32                             "\nS - max = " + mnmxS.getMax());
33     }
34 }

```

It is also possible to parametrize just a method (also static), not the whole class. We then specify the name of the type parameter (in angle brackets) just before the return type of the method in question:

```
access_specifier [static] <T> return_type name(parameter_list)
```

For example, in the following program, the class **MinMaxMeth** is *not* generic, however

the method **getMax** is: it may take an array of elements of any type, as long as this type implements **Comparable**:

Listing 58

GMI-MinMaxMeth/MinMaxMeth.java

```
1 class MinMaxMeth {
2     static <T extends Comparable<T>> T getMax(T[] arr) {
3         if (arr == null || arr.length == 0)
4             throw new IllegalArgumentException();
5         T max = arr[0];
6         for (int i = 1; i < arr.length; ++i)
7             if (arr[i].compareTo(max) > 0) max = arr[i];
8         return max;
9     }
10
11     public static void main(String[] args) {
12         int mxi = getMax(new Integer[]{3, -2, -7, 2});
13         // one may enforce type to be substituted for T;
14         // usually, as here, not needed, as the correct
15         // type will be inferred by the compiler anyway
16         String mxs = MinMaxMeth.<String>getMax(
17             new String[]{"A", "Z", "C"});
18
19         System.out.println("I - max = " + mxi);
20         System.out.println("S - max = " + mxs);
21     }
22 }
```

Section 5

Enum types

Enum types roughly correspond to enumerations in C++, but are implemented in a different way: they are all *objects*, in contrast to C++ where they are always backed by an integer type. They are very useful in situations when we need a type with only very limited number of possible values.

5.1 Basic definitions

Defining an enum we define a new type (just like defining a class). However, this type is somewhat special: limited number of *unmodifiable* enum constants (which are implemented as immutable objects) of this type is created when an enum class is loaded by the JVM and then it is impossible to create any more such objects. Therefore, we can say, that this type defines just a — usually small — set of constants. In fact, we know another example of type with only a few possible values: type **boolean** has only two — **true** and **false**. However, **boolean** is a primitive type, while enum constants are full-fledged objects.

Each of these constants has a fixed name, as is the case for **booleans**, and can be in fact a singleton of a *different* class.

Enums can be useful for defining types that by their very nature have only a small number of possible values: there are only two sexes, four seasons, seven Wonders of the World, four card suits and four Horsemen of the Apocalypse. Without enums, we could just assign numbers to them; for example, in a class or an interface we could write

```
final static int CLUBS    = 0,
                DIAMONDS  = 1,
                HEARTS     = 2,
                SPADES     = 3;
```

but this poses a lot of problems

- if a function takes a card suit as its parameter, it has to declare it as an **int**. But then nothing can prevent us from passing, for example, 129 as the argument, which wouldn't make any sense;
- of course, all functions operating on card suits could check if the value passed to them is in the range [0, 3], but then it would be hard to add a new suit (joker...) — we would have to correct the checking and interpretation in many places;
- in many situations we would have to remember the interpretation of the numbers; for example, printing just numbers as they are wouldn't be very informative — we would have to “convert” them manually into **Strings** or do something equivalent.

Let us look at an example:

Listing 59

CYG-Enum1/EnumEx1.java

```
1 public class EnumEx1 {
2
3     public enum Season {SPRING, SUMMER, FALL, WINTER};
```

```

4
5     public static void main (String[] args) {
6         Season[] seasons = Season.values();           //+1
7         for (Season s : seasons)
8             System.out.print(s + " ");               //+2
9         System.out.println();
10
11        System.out.println(Season.WINTER + " is the " +
12            (Season.WINTER.ordinal()+1) + "th season"); //+3
13        Season f = Season.valueOf("FALL");             //+4
14        System.out.println("FALL is " + f + "...");
15        System.out.println("Is f equal to FALL? " +
16            (f == Season.FALL));                       //+5
17        for (Season s : seasons)
18            System.out.print(german(s) + " ");
19        System.out.println();
20    }
21
22    private static String german(Season s) {
23        return switch (s) {                             //+6
24            case SPRING -> "Fr\u00fchling";
25            case SUMMER -> "Sommer";
26            case FALL   -> "Herbst";
27            case WINTER -> "Winter";
28        };
29    }
30 }

```

The program prints

```

SPRING SUMMER FALL WINTER
WINTER is the 4th season
FALL is FALL...
Is f equal to FALL? true
Fr\u00fchling Sommer Herbst Winter

```

We define an enum type **Season**. The definition is inside a class here, but this is not important; equally well we could have defined the enum (with **public** or default accessibility) in a separate file. This enum has exactly four values corresponding to four, and only four, objects: **SPRING**, **SUMMER**, **FALL**, **WINTER**. What can we do with type **Season** and its constants?

- static function **values()** returns an array of all (four in our case) constants of the enum (line `//+1`);
- method **toString()** is automatically overridden and returns the name of the enum constant (it is used in line `//+2`);
- we can call **ordinal()** on enum constants (remember that these are objects) and get their ‘index’ — starting from 0 and in the order as they were defined (`//+3`);
- static function **valueOf(String name)** returns enum constant named **name**; if there were no constant with this name, exception would have been thrown (`//+4`).

If we have two references to the same enum constant, they are exactly equal, i.e., they point to the same object, because each enum constant is represented by exactly one object. Therefore, to compare enum constant we don't need (although we can) to use method **equals**: just '==' or '!=' are safe and sufficient (//+5).

- as integer types and **Strings**, enum constants may be used in **switch** statements (//+6). In **case** clauses we don't have to use full names (like **Season.SPRING**), because the type of the selector (s in our case) is known to the compiler.

After compiling the above program

```
java> ls -l EnumExample1*
      EnumExample1.java
java> javac EnumExample1.java
java> ls -l EnumExample1*.class
      EnumExample1$1.class
      EnumExample1.class
      EnumExample1$Season.class
```

we can see that the compiler created *one* class for the **Season** type (which in our case was embedded in the main class, but this is not important here — anyway, a separate class has been created). The program prints

```
SPRING SUMMER FALL WINTER
WINTER is the 4th season
FALL is FALL...
Is f equal to FALL? true
Frühling Sommer Herbst Winter
```

The third line convinces us that indeed two references to **FALL** are exactly equal, that is they refer to the same object (and therefore can be compared by '==').

Enumerations are also comparable. If **e1** and **e2** are two enumerators of the same enumeration, then they can be compared in the usual way

```
e1.compareTo(e2)
```

The order is determined by the order in which enumerators are declared in the definition of enumeration.

In the following example there are two enumerations describing suits

Listing 60

CYE-SimpEnum/Suit.java

```
1 public enum Suit {CLUBS,DIAMONDS,HEARTS,SPADES};
```

and ranks

Listing 61

CYE-SimpEnum/Rank.java

```
1 public enum Rank {TWO,THREE,FOUR,FIVE,SIX,SEVEN,
2                 EIGHT,NINE,TEN,JACK,QUEEN,KING,ACE};
```

of playing cards. The class **Card** has two fields, both are enumerators (rank and suit); it also defines one static function:

Listing 62

CYE-SimpEnum/Card.java

```

1 public class Card {
2     private Rank rank;
3     private Suit suit;
4
5     public Card(Rank rank, Suit suit) {
6         this.rank = rank;
7         this.suit = suit;
8     }
9
10    public Rank getRank() { return rank; }
11    public Suit getSuit() { return suit; }
12
13    public static Card getHigher(Card c1, Card c2) {
14        if (c1.rank.ordinal() > c2.rank.ordinal())
15            return c1;
16        else if (c1.rank.ordinal() < c2.rank.ordinal())
17            return c2;
18        else if (c1.suit.ordinal() > c2.suit.ordinal())
19            return c1;
20        else
21            return c2;
22    }
23
24    @Override
25    public String toString() {
26        return rank + " of " + suit;
27    }
28 }

```

In **Main** we read data and use our classes

Listing 63

CYE-SimpEnum/Main.java

```

1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.print("Suits:");
6         for (Suit s : Suit.values())
7             System.out.print(" " + s );
8         System.out.print("\nRanks:");
9         for (Rank r : Rank.values())
10            System.out.print(" " + r);
11        System.out.println();
12
13        Scanner scan = new Scanner(System.in);
14        System.out.print("1. rank -> ");
15        String r1 = scan.next().toUpperCase();

```

```

16     System.out.print("1. suit -> ");
17     String s1 = scan.next().toUpperCase();
18     System.out.print("2. rank -> ");
19     String r2 = scan.next().toUpperCase();
20     System.out.print("2. suit -> ");
21     String s2 = scan.next().toUpperCase();
22
23     Rank rank1 = Rank.valueOf(r1);
24     Suit suit1 = Suit.valueOf(s1);
25     Rank rank2 = Rank.valueOf(r2);
26     Suit suit2 = Suit.valueOf(s2);
27
28     Card card1 = new Card(rank1,suit1);
29     Card card2 = new Card(rank2,suit2);
30
31     Card higher = Card.getHigher(card1,card2);
32     System.out.println("Card1 = " + card1 +
33                        ", Card2 = " + card2 +
34                        "\nHigher card: " + higher);
35 }
36 }

```

5.2 Fields, constructors and methods in enumerations

In the previous example, enum constants differ only in their names, so objects representing them were of the same type. However, one can define methods for them, and these methods can have different implementation for each constant separately. First, let us look at the following example:

Listing 64

CYH-EnumMet/EnumEx2.java

```

1 public class EnumEx2 {
2
3     enum Season {
4         SPRING {                                     //+1
5             @Override
6             public String toString() {return "Printemps";}
7         },
8         SUMMER {
9             @Override
10            public String toString() {
11                return "\u00c9t\u00e9";
12            }
13        },
14        FALL {
15            @Override
16            public String toString() {return "Automne";}
17        },

```

```

18         WINTER {
19             @Override
20             public String toString() {return "Hiver";}
21         }
22     };
23
24     public static void main (String[] args) {
25         Season[] seasons = Season.values();
26         for (Season s : seasons)
27             System.out.print(s + " ");           //+2
28         System.out.println();
29         for (Season s : seasons)
30             System.out.print(s.name() + " ");    //+3
31         System.out.println();
32     }
33 }

```

Here, for each constant separately, in braces just after their names (`//+1`), we override **toString** method from class **Object**. When we compile this program:

```

java> ls -1 EnumExample2*
EnumExample2.java
java> javac EnumExample2.java
java> ls -1 EnumExample2*.class
EnumExample2$1.class
EnumExample2.class
EnumExample2$Season$1.class
EnumExample2$Season$2.class
EnumExample2$Season$3.class
EnumExample2$Season$4.class
EnumExample2$Season.class

```

we can see that the compiler created distinct classes for each constant. This is quite obvious: they cannot be objects of the same class, as it would be impossible to have in one class several implementations of the same method **toString**! Now, when we print `s` (in line `//+2`), the overridden version of **toString** will be used. Still, however, the original ‘true’ name of enum constant may be retrieved by method **name()** (`//+3`), as we can see from the output

```

Printemps Été Automne Hiver
SPRING SUMMER FALL WINTER

```

Methods that we can define for enum constants are not limited to those inherited. We can define our own methods; moreover, we can add data fields and a constructor (but only one) as well!

In the program below we add two data fields: `desc` and `numOfMonths` (line `//+3`). We also define a standard constructor. Fields and constructors in enumerations are by definition **private**, so we don’t even need to declare them as such. Our constructor takes two arguments, so we supply them when defining the enum constants (line `//+1`):

Listing 65

CYI-EnumAbs/EnumEx3.java

```

1 public class EnumEx3 {
2     enum Season {
3         SPRING("nice",2) {                               //+1
4             @Override                                   //+2
5             public String getDesc() {
6                 return name() + ": " + desc;
7             }
8         },
9         SUMMER("hot",3) {
10             @Override
11             public String getDesc() {
12                 return desc + " in " + name();
13             }
14         },
15         FALL("so, so",4) {
16             @Override
17             public String getDesc() {
18                 return name() + "? Well, " + desc;
19             }
20         },
21         WINTER("cold",3) {
22             @Override
23             public String getDesc() {
24                 return name() + "! very " + desc;
25             }
26     }; // <-- semicolon after the last value!
27
28     // fields and constructors are private anyway
29     String desc; //+3
30     int numOfMonth;
31     Season(String d, int i) { //+4
32         desc = d;
33         numOfMonth = i;
34     }
35
36     public int getNumb() { return numOfMonth; } //+5
37     public abstract String getDesc(); //+6
38 };
39
40 public static void main (String[] args) {
41     for (Season s : Season.values())
42         System.out.println(s.getNumb() +
43             " months - " + s.getDesc());
44 }
45 }

```

There are also two methods. One of them, **getNumb** has the same common implementation for all objects (line //+5). However, the second one, **getDesc**, is in line //+5

only *declared* (as abstract) and implemented differently for each constant (line `//+2`). The program prints

```
2 months - SPRING: nice
3 months - hot in SUMMER
4 months - FALL? Well, so, so
3 months - WINTER! very cold
```

Objects representing enum constants are created once only, when the enum is loaded by the JVM, even before initialization of static members of the class, if there are any.

5.3 Enumarations implementing an interface

Enumerator can also implement interfaces. Let us look at an example:

Listing 66

CYK-EnumInterf/CompEnum.java

```
1  import java.util.Arrays;
2  import java.util.Comparator;
3
4  public class CompEnum {
5      public static void main(String[] args) {
6          String[] arr = {"Alice", "Sue", "Janet", "Bea"};
7
8          Arrays.sort(arr, StrCmp.ByLenAsc);
9          System.out.println(Arrays.toString(arr));
10
11         Arrays.sort(arr, StrCmp.ByLenDesc);
12         System.out.println(Arrays.toString(arr));
13
14         Arrays.sort(arr, StrCmp.ByLexAsc);
15         System.out.println(Arrays.toString(arr));
16
17         Arrays.sort(arr, StrCmp.ByLexDesc);
18         System.out.println(Arrays.toString(arr));
19     }
20 }
21
22 enum StrCmp implements Comparator<String> {
23     ByLenAsc( (s1, s2) -> s1.length() - s2.length()),
24     ByLenDesc((s1, s2) -> s2.length() - s1.length()),
25     ByLexAsc( (s1, s2) -> s1.compareTo(s2)),
26     ByLexDesc((s1, s2) -> s2.compareTo(s1)); // <- semicolon!
27     Comparator<String> cmp; // field
28     StrCmp(Comparator<String> c) { // constructor
29         cmp = c;
30     }
31     @Override
32     public int compare(String s1, String s2) {
33         return cmp.compare(s1, s2);
34     }
35 }
```

35 }

The enumeration **StrCmp** implements the **Comparator** interface. It has a field **cmp** of type **Comparator** which will be different for all enumeration constants: it is set in the constructor. When calling the constructor, we have to pass something which *is* a comparator — here, we use different lambdas. Objects of this enumeration can then be used wherever a comparator (of **Strings**) is expected, as we can see from the output of the program:

```
[Sue, Bea, Alice, Janet]
[Alice, Janet, Sue, Bea]
[Alice, Bea, Janet, Sue]
[Sue, Janet, Bea, Alice]
```

Introduction to collections

Collections represent... well, collections, i.e., aggregates of pieces of data of a given type organized in some form. Objects representing collections in Java have to implement the interface **Collection** (there is a different kind of collections, map, which implement different interface: **Map**). All collections are **iterable** (they implement also the **Iterable** interface) — the importance of this will soon become clear.

We already know interfaces. Generally speaking it's something like a class, but with declarations of (abstract) methods — without implementation (although, as we know, it can contain some *default* methods *with* implementation). Other classes may *implement* this interface by providing *definitions* of all the abstract methods. Any class may inherit from (extend) only one class, but can implement many interfaces.

Important:

1. Collections (and maps) are always collections of *references* (pointers) — *never* objects themselves!
2. Collections fall into two general categories: **Collections** and **Maps**.
3. Classes representing collections are generic — they are parametrized by the type of the elements they hold.
4. Their properties and functionality (API) is specified by a hierarchy of interfaces that they implement (i.e., they implement methods declared in these interfaces).

Very often we want a collection of data of a primitive type, like **int** or **double**. We cannot insert such data into any collection, because these are not references. However, for each such type, there is a class, objects of which serve as wrappers of such data: **Integer** for **ints**, **Double** for **doubles**, etc. Normally, we don't even have to create object of these wrapper classes ourselves — this will be done automatically: if a collection is declared as a collection of **Integers**, we can insert just **ints** and they will be automatically wrapped into objects of type **Integer** and put into this collection. Such automatic conversion of values of primitive types to objects is called **boxing**; the reverse operation is called **unboxing**.

When creating objects representing a collection, we should always specify the type of its elements; for maps there are two types to be specified: type of keys and of values; we do it using angle brackets, as we will see in the examples below.

6.1 Collections

The **Collection** interface contains the following methods, which therefore must be implemented in all concrete classes representing collections:

- **add** — adds an element;
- **addAll** — adds all elements of another collection;
- **clear** — removes all elements;
- **contains** — checks if an object belongs to this collection;
- **containsAll** — checks if all elements of another collection belong to this collection;
- **equals** — compares this collection with another one;
- **hashCode** — return hash code of the collection (overriding the method inherited from **Object**);

- **empty** — returns **true** if this collection is empty;
- **iterator** — returns an iterator associated with this collection (this is an implementation of the interface **Iterable**);
- **parallelStream** — returns parallel stream with this collection as its source (defaulted);
- **remove** — removes an element from this collection;
- **removeAll** — removes all elements from another collection from this collection;
- **removeIf** — removes all elements satisfying a given predicate (defaulted);
- **retainAll** — retains in this collection only elements from another collection;
- **size** — returns the number of elements in this collection;
- **splitIterator** — returns a split iterator associated with this collection (defaulted);
- **stream** — returns the stream with this collection as its source;
- **toArray** — returns an array containing all elements of this collection;

Some collections do not permit certain operations — then their implementations just throw an exception; we say that these operations (represented by methods) are then *optional*.

There are some subinterfaces of **Collection** that are more specific for various kinds of collections. The most important are

- **List**: lists represent collections of elements that are ordered (like elements of an array) — it makes sense to talk about element number 0, number 1, etc. Therefore, there are some methods which are not applicable to all collections but specific to lists: e.g., **get(i)** which returns element with the index specified, **indexOf(val)** which returns the index of an element with a given value, **add(i,val)** which adds new element at the position specified, etc. For all lists, adding a new element at the end is very efficient; at other locations — not necessarily so. A list may contain equal values at different positions. There are two main implementations of lists (concrete classes, *not* interfaces):
 - **ArrayList** — implementation is based on arrays. Adding elements *not* at the end may be very inefficient but access to all elements (by index) is almost immediate.
 - **LinkedList** — implementation is based on doubly-linked lists. Adding and removing may be fast, but access is slower.
- **Set**: sets represent collections of *unique* elements (no two elements are equal). There are two main implementations of sets:
 - **TreeSet** — implementation is based on red-black tree; the elements have a well defined order (therefore they have to be *comparable*), access to elements is fast (logarithmic).
 - **HashSet** — implementation is based on hashing technique — access to elements is even faster (constant time) but the order is unspecified.

6.2 Maps

Maps represent sets of *pairs* of objects: the first element of a pair is a **key** and the second is a **value** associated with this key (as elements of an array are associated with their indices, which therefore may be viewed as playing the rôle of keys). Types of keys and values may be, and usually are, different. As keys are used as “indices”, there may be no two equal keys in any map — they have to be unique (but the same value

may be associated with two different keys). All maps have to implement methods from **Map** interface; among others, these are:

- **put(key, val)** — adds a (key,value) pair to this map;
- **putIfAbsent(key, val)** — adds a pair to this map if the specified key is *not* already present (returning **null**); otherwise it does nothing and returns the value already associated with the key;
- **get(key)** — returns the value associated with a given key, or **null** if the map doesn't contain this key;
- **getOrDefault(key, defaultVal)** — returns the value associated with a given key, or defaultVal if the map doesn't contain this key;
- **clear** — removes all elements;
- **containsKey** — checks if there is a pair (the so called **entry**) in this map with a given key;
- **containsValue** — checks if there is a pair in this map with a given value (this is usually rather inefficient);
- **size** — returns the number of elements (entries) in this map;
- **isEmpty** — checks if the map is empty;
- **remove(key)** — removes the entry with a given key;
- **keySet** — returns a set of all keys;
- **values** — returns a collection of all values;
- **entrySet** — returns a set of entries, each of which has a key and a value accessible by methods **getKey** and **getValue**;
- etc.

There are two main implementations of maps:

- **TreeMap** — implementation is based on red-black tree of *keys*; the elements must have a well defined order. Therefore, *keys* have to be **Comparable**, or you have to pass a **Comparator** to the constructor. Access to elements is fast (logarithmic).
- **HashMap** — implementation is based on hashing technique — access to elements is even faster (constant time) but the order is unspecified.

6.3 Iterators

Iterators are objects representing a “view” of the elements of a collection and can yield on demand these elements, remembering which have already been yielded and knowing if there is anything that has not been returned yet. Iterators are also “generic”, so we should always specify the type of elements which they will return. The **Iterator** interface declares methods (here **T** is the type of elements)

```
boolean hasNext()
T next()
```

The second one returns the next element from those that have not been returned yet. The first one tells us if there is still an element that has not been returned by **next** (calling **next** when **hasNext** returns **false** should always trigger the exception **NoSuchElementException**).

There is the third operation which can be invoked on an iterators, **remove**: it removes the last element of the underlying collection that has just been returned by **next**. However, we don't have to implement it, because this operation is optional and

it already *has* a default implementation (which just throws an exception.) There is also a default (already implemented) method **forEachRemaining**.

The collection of elements which backs an iterator can be anything that implements **Iterable** interface. All “real” collections — classes from the standard library that implement **Collection** — *do* implement **Iterable**. **Iterable** declares just one abstract method:

```
Iterator<T> iterator()
```

(where **T** is the type of elements) which returns an iterator associated with a given collection implementing **Iterable**. In fact, this need not to be a “real” collection, it should only behave as one from the point of view of the iterator returned.

Let us consider an example — here object of type **IterableRange** plays the rôle of a collection, although there is no array or any other “true” collection involved: however, it implements **Iterable** and the iterator returned by its **iterator** method meets all the requirements of an iterator:

Listing 67

JIX-RangIter/RangIter.java

```

1 import java.util.Iterator;
2 import java.util.NoSuchElementException;
3
4 // main class for testing
5 public class RangIter {
6     public static void main (String[] args) {
7         Iterable<Integer> iterabl = new IterableRange(3,7);
8         Iterator<Integer> iter = iterabl.iterator();
9         while (iter.hasNext())
10             System.out.print(iter.next() + " ");
11
12         System.out.println("\nand now foreach:");
13         for (Integer i : new IterableRange(3,7))
14             System.out.print(i + " ");
15         System.out.println();
16     }
17 }
18
19 // objects of this class are 'iterable', i.e., they
20 // behave (at least to some extent) as collections
21 class IterableRange implements Iterable<Integer> {
22     private int a, b;
23     IterableRange(int a, int b) {
24         this.a = a;
25         this.b = b;
26     }
27     public Iterator<Integer> iterator() {
28         return new RangeIterator(a,b);
29     }
30 }
31
32 // object of this class behave like iterators
33 // traversing the IterableRange 'collections'

```

```

34 class RangeIterator implements Iterator<Integer> {
35     private int a, b;
36     private int curr;
37     RangeIterator(int a, int b) {
38         this.a = a;
39         this.b = b;
40         curr = a;
41     }
42     @Override
43     public boolean hasNext() {
44         return curr <= b;
45     }
46     @Override
47     public Integer next() {
48         if (!hasNext()) throw new NoSuchElementException();
49         return curr++;
50     }
51     // since Java 1.8 remove has a default implementation
52 }

```

All classes are here defined in one file for simplicity, it is generally not a recommended practice. Note that the class **RangeIterator** could have been defined inside **IterableRange** (see sec. 3.3). Note also the *for-each* (called also *ranged for*) loop used here in the **main** function:

```

for (Integer i : new IterableRange(3, 7))
    System.out.print(i + " ");

```

This form of loops works not only with standard collections, but in fact with anything that implements **Iterable**. Basically, the form

```

for (Type elem : iterable_object)
    do_something_with_elem

```

is by the compiler automatically translated into something like this

```

{
    Iterator<Type> it = iterable_object.iterator();
    while (it.hasNext()) {
        Type elem = it.next();
        do_something_with_elem
    }
}

```

It is quite common to create classes that are iterable and at the same time iterators. Implementation of the **iterator** method is then very simple — it just returns **this**. Let us rewrite the previous example in this way:

Listing 68

JIZ-IterIterable/IterIterable.java

```

1 import java.util.Iterator;
2 import java.util.NoSuchElementException;

```

```

3
4 // main class for testing
5 public class IterIterable {
6     public static void main (String[] args) {
7         for (Integer e : new IterableRange(3,11))
8             System.out.print(e + " ");
9         System.out.println();
10    }
11 }
12
13 // objects of this class are iterable and iterators!
14 class IterableRange implements Iterable<Integer>,
15                                Iterator<Integer> {
16     private int a, b, curr;
17     IterableRange(int a, int b) {
18         this.a = a;
19         this.b = b;
20         curr = a;
21     }
22     @Override
23     public Iterator<Integer> iterator() {
24         return this;
25     }
26     @Override
27     public boolean hasNext() {
28         return curr <= b;
29     }
30     @Override
31     public Integer next() {
32         if (!hasNext()) throw new NoSuchElementException();
33         return curr++;
34     }
35 }

```

As we can see, implementation is now much simpler and shorter.

6.4 Examples

Let us consider a few examples. First, lists and sets (i.e., collections implementing the **Collection** interface)

Listing 69

HUG-Lists/AList.java

```

1 import java.util.ArrayList;
2 import java.util.Collection;
3 import java.util.Collections;
4 import java.util.List;
5 import java.util.Set;
6 import java.util.HashSet;

```



```

7  import java.util.TreeSet;
8  import java.util.Iterator;
9
10 public class AList {
11     public static void main(String[] args) {
12         List<String> list = new ArrayList<>();
13         list.add("Sue");
14         list.add("Lea");
15         list.add("Ann");
16         list.add("Kim");
17         list.add("Lea");
18         list.add(1,"Amy"); // inefficient!
19
20         System.out.println("First 'Lea' under index " +
21                             list.indexOf("Lea"));
22         System.out.println("Last 'Lea' under index " +
23                             list.lastIndexOf("Lea"));
24         System.out.println("Does the list contain 'Sue': " +
25                             list.contains("Sue"));
26         System.out.println("Does the list contain 'Bea': " +
27                             list.contains("Bea"));
28         System.out.println("Size of list: " + list.size());
29
30         // traditional looping
31         System.out.print("With get(): ");
32         for (int i = 0; i < list.size(); ++i)
33             System.out.print(" " + list.get(i));
34         // `foreach' loop
35         System.out.print("\nAnother way: ");
36         for (String s : list)
37             System.out.print(" " + s);
38         // iterators
39         System.out.print("\nnow iterator:");
40         Iterator<String> iter = list.iterator();
41         while (iter.hasNext())
42             System.out.print(" " + iter.next());
43
44         // sorting and sublists
45         Collections.sort(list);
46         System.out.println("\nSorted:      " + list);
47         List<String> subl = list.subList(1,4);
48         System.out.println("Sublist:      " + subl);
49
50         // HashSet
51         Set<String> hSet = new HashSet<>(list);
52         System.out.println("Hash set:      " + hSet);
53         // TreeSet
54         Set<String> tSet = new TreeSet<>(list);
55         System.out.println("Tree set:      " + tSet);
56         System.out.println("Does the tSet contain 'Sue': " +

```

```

57                                     tSet .contains("Sue"));
58     tSet.add("Zoe");
59     tSet.remove("Lea");
60     System.out.println("tSet now:      " + tSet);
61 }
62 }

```

The program prints

```

First 'Lea' under index 2
Last  'Lea' under index 5
Does the list contain 'Sue': true
Does the list contain 'Bea': false
Size of list: 6
With get():   Sue Amy Lea Ann Kim Lea
Another way:  Sue Amy Lea Ann Kim Lea
now iterator: Sue Amy Lea Ann Kim Lea
Sorted:      [Amy, Ann, Kim, Lea, Lea, Sue]
Sublist:     [Ann, Kim, Lea]
Hash set:    [Ann, Sue, Lea, Amy, Kim]
Tree set:    [Amy, Ann, Kim, Lea, Sue]
Does the tSet contain 'Sue': true
tSet now:    [Amy, Ann, Kim, Sue, Zoe]

```

Note, that iteration over a **TreeSet** collection gives its elements in alphabetical order, what corresponds to the natural order of **Strings**. This is not true for a **HashSet**. Note also various forms of iterations over **Lists**, in particular the *for-each* loop (that we already used for arrays).

Now maps (i.e., collections implementing the **Map** interface, but not **Collection**):

Listing 70

HUK-SimpleMap/ASimpleMap.java

```

1  import java.util.HashMap;
2  import java.util.Iterator;
3  import java.util.Map;
4  import java.util.Set;
5  import java.util.TreeMap;
6
7  public class ASimpleMap {
8      public static void main(String[] args) {
9          Map<String,Integer> map = new TreeMap<>();
10         map.put("Sue",167);
11         map.put("Ann",173);
12         map.put("Lea",170);
13         map.put("Kim",173);
14
15         Iterator<String> iter = map.keySet().iterator();
16         while (iter.hasNext()) {
17             String key = iter.next();
18             int     val = map.get(key); // auto(un)boxing

```

```

19         System.out.print(key + ": " + val + " ");
20         if (val == 170) iter.remove();
21     }
22
23     System.out.print("\nWe can print a map: " + map);
24
25     // returns null, or the value if present
26     map.putIfAbsent("Lea",169);
27     // returns old value or null if not present
28     map.replace("Lea",170);
29
30     System.out.print("\nIs there a key 'Lea'? " +
31                     map.containsKey("Lea"));
32     System.out.print("\nIs there a key 'Zoe'? " +
33                     map.containsKey("Zoe"));
34     // inefficient!
35     System.out.println("\nIs any girl 170 cm tall? " +
36                       map.containsValue(170));
37
38     Integer was = map.remove("Ann");
39     if (was != null)
40         System.out.println("Ann removed, she was " +
41                             was + " cm tall");
42     was = map.remove("Zoe");
43     if (was == null)
44         System.out.println("There was no 'Zoe!'");
45
46     System.out.println("getOrDefault: 'Zoe'->" +
47                       map.getOrDefault("Zoe",-1));
48     System.out.println("getOrDefault: 'Lea'->" +
49                       map.getOrDefault("Lea",-1));
50
51     System.out.print("Iterating over 'keySet': ");
52     for (String s : map.keySet())
53         System.out.print(s + "->" + map.get(s) + " ");
54
55     System.out.print("\nMore efficient way: ");
56     for (Map.Entry<String,Integer> e : map.entrySet())
57         System.out.print(e.getKey() + "->" +
58                           e.getValue() + " ");
59     System.out.println();
60 }
61 }

```

The program prints

```

Ann: 173  Kim: 173  Lea: 170  Sue: 167
We can print a map: {Ann=173, Kim=173, Sue=167}
Is there a key 'Lea'? true
Is there a key 'Zoe'? false

```

```

Is any girl 170 cm tall? true
Ann removed, she was 173 cm tall
There was no 'Zoe'!
getOrDefault: 'Zoe' -> -1
getOrDefault: 'Lea' -> 170
Iterating over 'keySet': Kim->173 Lea->170 Sue->167
More efficient way: Kim->173 Lea->170 Sue->167

```

Note, that **Maps** themselves are *not* iterable; however, a map's key set (returned by the **keySet** method), being a **Set**, i.e., a **Collection**, *is* iterable. The same applies to the **Set** of type **Map.Entry** objects returned by the **entrySet** method — each of such objects represents one (key, value) pair, of which both components may be accessed separately by the methods **getKey** and **getValue**.

6.5 Importance of equal and hashCode methods

Class **Object** defines — among others (in particular

```
public String toString()
```

that we already know) two other important methods:

- **public boolean equals(Object ob)** — which is used to check if two objects are, according to some criterion, equal: for two references **ob1** and **ob2**, the expression **ob1.equals(ob2)** compares the objects and yields **true** or **false**. But according to what criterion? In class **Object** there is no data to be compared, so the default implementation just compares addresses of the two objects. Very often, this is *not* what we want. When we have two objects of class **Person** and these persons have identical names, dates of birth, passport numbers etc., we rather want to consider the two objects as representing exactly the same person, in other words we want to consider these two objects equal. To get this behavior, we thus have to redefine (override) **equals** in our class.

The **equals** method should implement an equivalence relation on non-null object references. This means that for any non-null references **a**, **b** and **c** **a.equals(a)** should always be **true** (reflexivity), **a.equals(b)** should have the same value (**true** or **false**) as **b.equals(a)** (symmetry), and if **a.equals(b)** and **b.equals(c)** are **true** then **a.equals(c)** must also be **true** (transitivity).

Reflexivity and symmetry are usually obvious, but transitivity — not always. Suppose, we consider two two-element sets equal if they have at least one common element. Then $A = \{a, b\}$ is equal to $B = \{b, c\}$ and B is equal to $C = \{c, d\}$, but A and C have no common element and are not equal.

- **public int hashCode()** — which is used to calculate the so called **hash code** of an object. This is necessary if we want to put objects of our class in collections implemented as hash tables (e.g., **HashSet** or keys in a **HashMap**). The implementation of **hashCode** method from the **Object** class uses just the address of the object to calculate its hash code. However, very often this is not desirable as it would lead to situations when two objects that are considered equal (according to **equals**) have different hash codes: as a consequence the collections of such objects would be invalidated. Therefore, we have to override also this method remembering to do it consistently: whenever two objects are equal according to **equal**, their hash codes should be exactly the same.

The following example illustrates **toString** and **equals** methods:

Listing 71

AAR-EquToString/EquToString.java

```
1 public class EquToString {
2     public static void main(String[] args) {
3         PersonGood johny = new PersonGood("John",1980);
4         PersonGood john = new PersonGood("John",1980);
5         PersonBad billy = new PersonBad("Bill",1980);
6         PersonBad bill = new PersonBad("Bill",1980);
7
8         if (johny.equals(john))
9             System.out.println("johny == john");
10        else
11            System.out.println("johny != john");
12
13        if (billy.equals(bill))
14            System.out.println("billy == bill");
15        else
16            System.out.println("billy != bill");
17
18        System.out.println("johny: " + johny);
19        System.out.println("billy: " + billy);
20    }
21 }
22
23 class PersonBad {
24     private String name;
25     private int byear;
26     PersonBad(String n, int y) {
27         name = n;
28         byear = y;
29     }
30     public String getName() { return name; }
31     public int getYear() { return byear; }
32 }
33
34 class PersonGood {
35     private String name;
36     private int byear;
37     PersonGood(String n, int y) {
38         name = n;
39         byear = y;
40     }
41
42     public String getName() { return name; }
43     public int getYer() { return byear; }
44
45     @Override
46     public String toString() {
47         return name + "(" + byear + ")";
48     }
49 }
```

```

49
50     @Override
51     public boolean equals(Object ob) {
52         if (ob == null || getClass() != ob.getClass()) return false;
53         PersonGood p = (PersonGood)ob;
54         return name.equals(p.name) && byear == p.byear;
55     }
56 }
57

```

As we can see, with **toString** and **equals** methods *not* overridden (in class **PersonBad**), the program works but uses the versions of these methods from class **Object**. To get the expected results, we have to override these methods, as we did in class **PersonGood**.

The next example illustrates the effect of overriding the **hashCode** method:

Listing 72

HUM-HashEquals/Person.java

```

1  public class Person {
2
3      private String name;
4      private String idNumber;
5
6      public Person(String name, String idNumber) {
7          this.name = name;
8          this.idNumber = idNumber;
9      }
10
11
12
13     @Override
14     public boolean equals(Object other) {
15         if (other == null ||
16             getClass() != other.getClass()) return false;
17         Person p = (Person)other;
18         return idNumber.equals(p.idNumber) &&
19             name.equals(p.name);
20     }
21
22
23
24     @Override
25     public int hashCode() {
26         return 17*name.hashCode() + idNumber.hashCode();
27     }
28
29
30     @Override
31     public String toString() {
32         return name + "(" + idNumber + ")";

```

```
33     }  
34 }
```

with **main** as below

Listing 73

HUM-HashEquals/AHash.java

```
1  import java.util.HashMap;  
2  import java.util.Map;  
3  
4  public class AHash {  
5      public static void main(String[] args) {  
6          Map<Person,String> map = new HashMap<>();  
7  
8          map.put(new Person("Sue","123456"),"Sue");  
9  
10         // new object, but should be equivalent to  
11         // the one which has been put into the map  
12         Person sue = new Person("Sue","123456");  
13  
14         if (map.containsKey(sue))  
15             System.out.println(sue + " has been found");  
16         else  
17             System.out.println(sue + " has NOT been found");  
18     }  
19 }
```

As can be easily checked, the program will print

Sue(123456) has been found

only when both **hashCode** and **equals** are consistently overridden in the class of keys of the map (i.e., **Person**).

Streams

7.1 Introduction

Streams provide an abstraction for dealing with collections of values (which do not have to correspond to ‘real’ collections) and specifying what you want to be done, leaving the details of how to do it to the library. For example, you can create a stream of object of type **Person** and say something like *select only women of age in the range [35, 60] and form a map with country of origin as keys and lists of women form these countries as values*. Or, having an infinite (*sic!*) stream of **doubles**, you can say *select only positive numbers, round them to integers, calculate their squares, take 1000 first elements and give me their arithmetic average*. In addition, you can often say something like *take my stream, do this and this and try to do it using as many separate threads as necessary to ensure maximum efficiency*.

Streams can be created from collections, arrays, or using generators or iterators. However, themselves, they are *not* collections. Rather, they represent streams of individual pieces of data that can be processed one by one, not necessarily storing anywhere and anytime all of them together.

- Usually, streams do not store their elements anywhere, they take it one by one from a source. However, for some operations they have to store the elements internally, for example in order to sort them.
- Streams do not modify the source of their elements, they can transform them and yield transformed values in a new stream.
- Operations on streams are ‘lazy’, i.e. only those operation that are needed to get the desired result are actually executed.

In order to use streams, you have to

- create a stream — you can require it to be parallelized if it makes sense for a problem at hand;
- apply zero, one or more *intermediate operations*; each of them yields another stream to which one can apply a next intermediate operation, thus forming the so called *pipeline*;
- apply a *terminal operation* that produces a result. Only a terminal operation triggers all the other (intermediate) operations; before that no processing takes place. This is why streams can be lazy — when it is known what result is expected, it is known which operations are necessary and which are not. After applying a terminal operation, the stream is deemed ‘consumed’; its is closed and cannot be reused.

Some operations, both intermediate and terminal, can be *short-circuiting*: an intermediate operation is short-circuiting if for an infinite stream it may produce a finite stream, a terminal operation is short-circuiting if for an infinite stream it may terminate in finite time. Of course, for infinite streams, there must be at least one short-circuiting operation in the whole pipeline of operations.

Intermediate operations can be stateless or stateful. Most of them are stateless — elements are processed one by one and to process a given element no knowledge of elements seen previously, or those to be seen later, is needed. Some operations, however,

are stateful — some information on the previous elements has to be remembered. There are four such operations in the library: **sorted**, **distinct**, **limit** and **skip**. For obvious reasons, the stateful operations cannot be applied to infinite streams and are less efficient.

In the examples below, we will use references to methods, which can be used instead of lambdas — they are covered in more detail in sec. 8 on p. 109.

We will also refer to functional interfaces that are already defined in the library: more about them in sec. 9 on p. 112.

7.2 Creating streams

Streams can be created from any collection by invoking its **stream** method

```
collection.stream()
```

which produces a stream of type **Stream<T>**, where **T** is the type of elements of the collection. There is also a static function **of** in the **Stream** class. It takes any number of arguments, or an array, and creates a stream:

```
Stream<String> s1 = Stream.of("Alice", "Cindy", "Kate");
Stream<String> s2 = Stream.of(new String[]{"A", "B"});
```

Another static function of **Stream** is **generate**. It takes a **Supplier** and produces a stream by using the supplier to create (potentially infinite) stream of values

```
Stream<Double> d = Stream.generate(Math::random);
```

One can also use **iterate** which takes a ‘seed’ and a **UnaryOperator** and repeatedly applies the operator to the previous element with seed as the first one, so the resulting stream will be

```
(seed, op.apply(seed), op.apply(op.apply(seed)), ...)
```

For example:

```
Stream.iterate(1, n -> 2*n).limit(7).forEach(System.out::println);
```

will print the sequence (1, 2, 4, 8, 16, 32, 64); note that we had to limit the number of elements in the stream, as **iterate** generates potentially infinite sequence of numbers.

Functions **generate** and **iterate** may also be used with primitive types; the type of the stream will then be **DoubleStream**, **LongStream** or **IntStream**.

Streams of primitive type, **IntStream** and **LongStream** can also be created by calling **range** and **rangeClosed**:

```
IntStream s = IntStream.range(int startIncl, int endExcl)
IntStream s = IntStream.rangeClosed(int startIncl, int endIncl)
```

which return an ordered stream of **ints** from **startIncl** (inclusive) to **endExcl** (exclusive) for the first form or to **endIncl** (inclusive) in the second form, by an incremental step of 1 (and analogously for **longs**).

Methods creating streams have also been added to some other classes, for example **Files** from *java.nio.file*. Its static method **lines** yields a stream of lines of a file as **Strings**:

```
try (Stream<String> str = Files.lines(path, charset)) {
    // str is a stream of lines as strings
}
```

We use try-with-resources here to be sure that when the stream is closed, the underlying file is also closed (if your `charset` is UTF-8, it can be omitted, as this is the default).

There is also, in class `Pattern` from `java.util.regex`, the method `splitAsStream` which yields a stream of `Strings` resulting from splitting a text (actually, a `CharSequence`) with a given regex specifying the separator:

```
Stream<String> words =
    Pattern.compile("\\P{L}+").splitAsStream(text);
```

will yield a stream of words from `text` (separated by non-empty sequences of non-letters).

You can find examples of stream creation in listings that follow (in particular, Listing 76 on page 99).

7.3 Intermediate operations

Let us enumerate the most useful intermediate operations (implemented as methods of class `Stream` from the package `java.util.stream`). In what follows, the symbol `T` denotes the type of elements in a stream on which the methods are invoked (its type is therefore `Stream<T>`), while `R` stands for another type, where applicable.

- `Stream<T> filter(Predicate<T> pred)` — produces a new stream where only elements that satisfy the predicate from the original stream are retained. Examples: Listing 75, Listing 80, Listing 81, Listing 83.
- `Stream<R> map(Function<T,R> fun)` — applies the function to each element of the input stream and produces a new stream of values obtained. There are similar operations with a primitive type instead of `R`: `mapToInt`, `mapToLong` and `mapToDouble`. Examples: Listing 74, Listing 75, Listing 76, Listing 77, Listing 81, Listing 82.
- `Stream<T> distinct()` — produces the same stream as the input one but with all duplicates removed (they need not to be on subsequent positions). To compare elements, method `equal` is used. Stateful (and rather expensive) operation. Examples: Listing 76, Listing 83.
- `Stream<R> flatMap(Function<T,Stream<R>> fun)` — applies `fun` to all elements to get stream of streams; then ‘flattens’ these streams into one stream of all elements from the individual streams. There are specialized versions for primitive types instead of `R`: `flatMapToInt`, `flatMapToLong` and `flatMapToDouble` which return `IntStream`, `LongStream` and `DoubleStream`, respectively. Example: Listing 83.
- `Stream<T> sorted(Comparator<T> cmp)` — yields the same stream but sorted by using the given comparator `cmp`. There is also version without any arguments — then the natural order of type `T` is used. Stateful operation. See also the documentation of `Comparator` from `java.util` for various ways of creating comparators; an example is shown in Listing 83. Other examples include Listing 76 and Listing 81.
- `Stream<T> peek(Consumer<T> cons)` — returns the same stream but executes `cons` on each element ‘on the fly’. Used often for debugging. Examples: Listing 81, Listing 74, Listing 75.

- `Stream<T> limit(long lim)` — returns the same stream but limited to at most `lim` first elements. Stateful, short-circuiting operation. Example: Listing 74.
- `Stream<T> skip(long skip)` — returns the same stream but with first `skip` elements suppressed. Stateful operation.

7.4 Terminal operations

Zero, one or more transformations by intermediate operations must be followed by exactly one final (terminal) operation; otherwise no operation at all would be actually executed as this is a terminal operation that forces the execution of all (lazy) operations that precede it. Terminal operation yields a result — this can be one value (in particular, a number) or a collection (or array) of elements.

Many terminal operations can be created by stream method `collect(Collector)`, where a collector can be chosen from the rich set of ready-to-use collectors returned by static factory methods defined in class `Collectors`.

There are several terminal operations yielding a number as their result. For example (`T` denotes the type of elements in the stream, `NumType` stands for `Int`, `Long` or `Double`):

- `long count()` — returns the number of elements in the input stream;
- `Optional<T> min(Comparator cmp), Optional<T> max(Comparator cmp)` — return minimum and maximum elements of the stream using a provided comparator. For primitive-type streams no comparator is needed and the return type is `OptionalInt`, `OptionalLong` or `OptionalDouble`. To get minimum or maximum element, one can also use `collect` with a collector returned by invoking `minBy(Comparator)` or `maxBy(Comparator)` from class `Collectors`. The optional returned is empty, if stream was empty.
- `NumType sum()` — returns the sum of elements for primitive-type streams. Similarly, a collector returned by `summingType(ToTypeFunction)` can be used for non-primitive type (`Type` is `Int`, `Long` or `Double`). See examples in Listing 76 and Listing 81.
- `Optional<Double> average()` — returns the mean (arithmetic average) for streams of primitive types. A collector returned by `averagingType(ToTypeFunction)` can be used for non-primitive types (`Type` is `Int`, `Long` or `Double`).

For primitive streams, there are also terminal operations `summaryStatistics`, which return an object of type `TypeSummaryStatistics`, where `Type` is `Int`, `Long` or `Double`. This object may then be queried for the number of elements, their sum, average, minimum and maximum. For non-primitive types one can use a collector returned by `summarizingType(ToTypeFunction)`. Very often, the `ToTypeFunction` function will be just the reference to a method returning a number. See an example in Listing 77.

Another group of terminal operations are those returning collections (or arrays). Let us mention some of them

- `Object[] toArray()` — returns all elements of the stream as an array (of type `Object[]`). See example in Listing 82.
- `R[] toArray(IntFunction<R[]> gen)` — returns an array of type `R[]`; `gen` is a function taking an `int` and creating an array of this size. Most often this will be just the reference to an array constructor `R[]::new`. See examples in Listing 75 and Listing 80.

- collector returned by `groupingBy(Function<T,K> keyExtr)` — produces a map of type `Map<K,List<T>>`. It applies the function `keyExtr` to each element of the stream and treats the obtained value as a key in the resulting map; lists of elements yielding the same key will be the values of the map (there are others, very powerful versions of this collector). See simple examples in Listing 78 and Listing 79.

A very useful terminal operation just ‘consumes’ the stream

```
void forEach(Consumer<T> cons)
```

by invoking `cons` on each element; a very common example is just printing:

```
stream.forEach(System.out::println).
```

One can also create a `String` by concatenating strings obtained from subsequent elements of a stream; such a collector can be created by factory method

```
String stream.collect(Collectors.joining())
```

(a desired delimiter may also be passed as the argument). See examples in Listing 76, Listing 77, Listing 81 and Listing 83.

There are also short-circuiting operations taking a `Predicate` and returning `boolean` — `allMatch` (whether all elements satisfy the given predicate), `anyMatch` (whether at least one satisfies it) and `noneMatch` (whether all elements do not satisfy the predicate). They are short-circuiting because the stream is deemed consumed (and processing stops) when the result is already known. For example, in the case of `allMatch`, when an element *not* satisfying the predicate is encountered, the answer is `false` and cannot change, so the processing stops. See example in Listing 76.

A very important and versatile reduction of a stream to a single value can be obtained by invoking `reduce` (the operation performed by `reduce` is called *left fold*). The operation has a few variants. The basic one looks like this

```
T reduce(T iden, BinaryOperator<T> acc)
```

Here, `acc` is a `BinaryOperator<T>` acting on two values of type `T` (type of elements of the stream). First the operator is applied to `iden` and the first element of the stream, then on the result and the second element, than again on the result and the third, and so on. Basically, this is equivalent to

```
T result = iden;
for (T e : elements of the stream)
    result = acc.apply(result, e)
return result;
```

There are two important conditions, though:

- `iden` must be an identity of the operation, i.e.,
`apply(iden,e)`
must return `e` (like number 0 for addition or 1 for multiplication);
- operator `apply` must be associative, i.e.,
`apply(apply(e,f),g) = apply(e,apply(f,g))`
must always hold.

Let us suppose that we have a stream `str` of `Integers`. Then we could use `reduce` to find the sum or product of all elements, or their count, like this

```

    // sum
Integer a = str.reduce(0, Integer::sum);
    // product
Integer a = str.reduce(1, (a,e) -> a*e);
    // count
Integer a = str.reduce(0, (a,e) -> a+1);

```

There are also other versions of **reduce** (without **iden**, but returning **Optional** — see documentation).

Streams can be parallelized. You either get a parallel stream directly from a collection

```
Stream<T> parallelStream = collection.parallelStream();
```

or parallelize an existing stream

```
Stream<T> parallelStream = sequentialStream.parallel();
```

You can also make a parallel stream sequential

```
Stream<T> sequentialStream = parallelStream.sequential();
```

Parallel streams allow the compiler to divide operations on the stream into parts that can be executed on different threads. Of course, the final result will have to be somehow combined from the partial results — this is not always trivial! It is your responsibility to ensure that operations performed on different threads do not lead to data races or deadlocks.

7.5 Examples

The example below uses **limit**, **peek**, **map** and **forEach** terminal operation. It demonstrates the ‘laziness’ of streams: only those elements which are necessary to get the result are actually processed. Also, a reference to a constructor is used:

Listing 74

LDC-Lazy/Lazy.java

```

1 import java.util.stream.Stream;
2 import java.io.File;
3
4 public class Lazy {
5     public static void main (String[] args) {
6         Stream.of("Alice", "Bella", "Cecilia", "Dorothy")
7             .peek(e -> System.out.print("peek: " + e + "; "))
8             .map(s -> s + ".txt")
9             .map(File::new)
10            .limit(2)
11            .forEach(f -> System.out.println(f + " exists? " +
12                (f.exists() ? "Yes" : "No") ));
13    }
14 }

```

The output is interesting:

```
peek: Alice; Alice.txt exists? No
peek: Bella; Bella.txt exists? Yes
```

As we can see, the first element ("Alice") started its "journey" and went all the way down to the **forEach** before the next element ("Bella") even started! After two first elements ("Alice" and "Bella") finished, the **limit(2)** "says" *that's enough* and no other element even starts its journey along the chain of operations — "Cecilia" and "Dorothy" don't even reach **peek**! This nicely illustrates the laziness of streams: only what is necessary to get the final result will be really executed.

The example below demonstrates the function **Files.lines** which creates a stream of lines of a given file which then can be transformed and reduced to a desired result (in this case, a list). It also demonstrates method references, as well as methods **filter**, **peek**, **map** and **toList** and **forEach** terminal operations.

Listing 75

LDF-StreamGrep/StreamGrep.java

```

1 import java.util.List;
2 import java.io.IOException;
3 import java.nio.file.Files;
4 import java.nio.file.Paths;
5 import java.util.stream.Collectors;
6 import java.util.stream.Stream;
7 import static java.nio.charset.StandardCharsets.UTF_8;
8
9 public class StreamGrep {
10     public static void main(String[] args) {
11         List<String> list = null;
12         try (Stream<String> lines =
13             Files.lines(Paths.get("StreamGrep.java"),
14                 UTF_8)) {
15             String substr = "String";
16             list = lines
17                 // Predicate expected
18                 .filter(s -> s.indexOf(substr) >= 0)
19                 // Consumer expected
20                 .peek(System.out::println)
21                 .collect(Collectors.toList());
22         } catch (IOException e) { return; }
23         System.out.println("and now the list...");
24         list.stream()
25             // reference to method
26             .map(String::toUpperCase)
27             .forEach(System.out::println);
28     }
29 }
```

The program prints

```
public static void main(String[] args) {
    List<String> list = null;
```

```

        try (Stream<String> lines =
            String substr = "String";
            .map(String::toUpperCase)
and now the list...
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {
        LIST<STRING> LIST = NULL;
        TRY (STREAM<STRING> LINES =
            STRING SUBSTR = "STRING";
            .MAP(STRING::TOUPPERCASE)

```

A stream of lines of a file can also be obtained from **BufferedReader**, as the example below demonstrates. It also shows streams of primitive type (e.g., **int**), various ways of creating streams, functions **sorted**, **distinct**, **filter**, **map**, **mapToInt**, **peek** and **sum**, **joining**, **allMatch** and **forEach** terminal operations. The last part of the program shows how to obtain a stream from a regex.

Listing 76

LDD-StreamMisc/StreamMisc.java

```

1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.nio.charset.StandardCharsets;
4  import java.nio.file.Files;
5  import java.nio.file.Paths;
6  import java.util.ArrayList;
7  import java.util.Arrays;
8  import java.util.List;
9  import java.util.stream.Collectors;
10 import java.util.stream.IntStream;
11 import java.util.stream.Stream;
12 import java.util.regex.Pattern;
13
14 public class StreamMisc {
15     public static void main(String[] args) {
16         System.out.println("*From an array...");
17         String[] ws = {"To", "be", "or", "not", "to", "be"};
18         Stream.of(ws)
19             .map(String::toLowerCase)
20             .distinct()
21             .sorted()
22             .forEach(e -> System.out.print(e + " "));
23         System.out.println();
24
25         System.out.println("*From varargs...");
26         System.out.println(
27             Stream.of("To", "be", "or", "not", "to", "be")
28                 .collect(Collectors.joining(" - "))
29         );
30
31         System.out.println("*From a collection...");
32         List<String> list = Arrays.asList(

```



```

33         "1","10","100","1000","10000","100000");
34     System.out.println("Sum = " +
35         list.stream().mapToInt(Integer::parseInt).sum()
36     );
37
38     // generating a stream by iterating a unary
39     // function starting from a given seed
40     System.out.println("*From a generator...");
41     ArrayList<Integer> arri = new ArrayList<>();
42     IntStream.iterate(17, n -> n%2 == 0 ? n/2 : 3*n+1)
43         .peek(arri::add)           // arri in closure,
44         .allMatch(n -> n != 1);    // allMatch is short-
45                                   // circuited, so will
46     System.out.println(arri); // stop iteration!
47
48     System.out.println("*Lines of a file as stream...");
49     try (BufferedReader br = Files.newBufferedReader(
50         Paths.get("StreamMisc.java"),
51         StandardCharsets.UTF_8)) // default
52     {
53         br.lines()
54             .filter(e -> e.contains("collect"))
55             .forEach(System.out::println);
56     } catch (IOException never_ignore_exceptions) { }
57
58     System.out.println("*From a regex...");
59     String s = "a is 1, b=3 and c:7 X";
60     System.out.println("Sum of extracted numbers is " +
61         Pattern.compile("\\d+")
62             .splitAsStream(s)
63             .filter(e -> e.length() > 0)
64             .mapToInt(Integer::parseInt).sum());
65     }
66 }

```

The program prints

```

*From an array...
be not or to
*From varargs...
To - be - or - not - to - be
*From a collection...
Sum = 111111
*From a generator...
[17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]
*Lines of a file as stream...
    .collect(Collectors.joining(" - "))
    System.out.println("*From a collection...");
    .filter(e -> e.contains("collect"))
*From a regex...

```


Sum of extracted numbers is 11

Primitive-type streams are also demonstrated in the example below; note the **summaryStatistics** terminal operation and the **generate** supplier. Also, functions **sorted**, **limit** and **map** are used here (as well as method references).

Listing 77

LDG-MethRefStr/MethRefStr.java

```

1  import java.util.Random;
2  import java.util.stream.DoubleStream;
3  import java.util.stream.Collectors;
4  import java.util.stream.Stream;
5
6  public class MethRefStr {
7      public static void main(String[] args) {
8          Random r = new Random(); // effectively final
9          System.out.println(
10              // Supplier of double's expected
11              DoubleStream.generate(r::nextGaussian)
12              // we want just ten million numbers
13              .limit(10_000_000)
14              // reduction to DoubleSummaryStatistics
15              .summaryStatistics()
16              // arithmetic average of all numbers
17              .getAverage());
18
19          System.out.println(
20              Stream.of(new Person("C"), new Person("A"),
21                      new Person("D"), new Person("B"))
22                      // Function<Person,otherType> expected
23                      .map(Person::getName)
24                      .sorted()
25                      // Function<String,otherType> expected
26                      .map(String::toLowerCase)
27                      // reduction to a single String
28                      .collect(Collectors.joining("-")));
29
30          Thread t = new Thread(MethRefStr::fibos);
31          t.start();
32          try {
33              t.join();
34          } catch (InterruptedException ignore) { }
35      }
36
37      public static void fibos() {
38          StringBuilder sb = new StringBuilder("0, 1");
39          int a = 0, b = 1;
40          for (int i = 0; i < 8; ++i) {
41              b += a;
42              a = b - a;

```

```

43         sb.append(", " + b);
44     }
45     System.out.println(sb);
46 }
47 }
48
49 class Person {
50     private String name;
51     public Person(String n) { name = n; }
52     public String getName() { return name; }
53 }

```

The program prints (the first number may be different)

```

-1.9002508699231762E-4
a-b-c-d
0, 1, 1, 2, 3, 5, 8, 13, 21, 34

```

The next example demonstrates the use of, extremely useful, **groupingBy** terminal operation. It comes in many variants; below, the simplest of them is used. It takes a function, which applied to elements of the stream will yield a value which will be then used as the key of the map: values of this map will be lists of elements that yield this key:

Listing 78

LDE-Grouping/Grouping.java

```

1  import java.util.Arrays;
2  import java.util.List;
3  import java.util.stream.Collectors;
4
5  public class Grouping {
6      public static void main (String[] args) {
7          List<Person> list = Arrays.asList(
8              new Person("John", "UK"),
9              new Person("Mary", "US"),
10             new Person("Xue", "CH"),
11             new Person("Kate", "UK"),
12             new Person("Janek", "PL"),
13             new Person("Cindy", "US"),
14             new Person("Bao", "CH"),
15             new Person("Kasia", "PL")
16         );
17
18         // collect gives Map<String,List<Person>>
19         // groupingBy expects Function...
20         list
21             .stream()
22             .collect(Collectors.groupingBy(Person::getCountry))
23             .entrySet()
24             .stream()

```

```

25         .forEach(e -> System.out.println(e.getKey() +
26             " -> " + e.getValue()));
27     }
28 }
29
30 class Person {
31     private final String name;
32     private final String country;
33     public Person(String n, String c) {
34         name = n; country = c;
35     }
36     public String getName()    { return name;    }
37     public String getCountry() { return country; }
38     @Override
39     public String toString() {
40         return name + " (" + country + ")";
41     }
42 }

```

The program prints

```

CH -> [Xue (CH), Bao (CH)]
UK -> [John (UK), Kate (UK)]
PL -> [Janek (PL), Kasia (PL)]
US -> [Mary (US), Cindy (US)]

```

A very similar example is given below: here, we add the second argument (a collector) to the **groupingBy** function. This second collector determines what to do with elements corresponding to the same key — here, we just count them

Listing 79

LDJ-GroupCount/GroupCount.java

```

1  import java.util.Arrays;
2  import java.util.List;
3  import java.util.stream.Collectors;
4
5  public class GroupCount {
6      public static void main (String[] args) {
7          List<Person> list = Arrays.asList(
8              new Person("John", "UK"), new Person("Ewa", "PL"),
9              new Person("Mary", "US"), new Person("Xue", "CH"),
10             new Person("Kate", "UK"), new Person("Jane", "UK"),
11             new Person("Janek", "PL"), new Person("Cindy", "US"),
12             new Person("Bao", "CH"), new Person("Kasia", "PL")
13         );
14
15         // collect gives Map<String, Long>
16         list
17             .stream()
18             .collect(Collectors.groupingBy(

```

```

19         Person::getCountry,
20         Collectors.counting()))
21     .entrySet()
22     .stream()
23     .forEach(e -> System.out.println(e.getKey() +
24                                     " -> " + e.getValue()));
25 }
26 }
27
28 class Person {
29     private final String name;
30     private final String country;
31     public Person(String n, String c) {
32         name = n; country = c;
33     }
34     public String getName()    { return name;    }
35     public String getCountry() { return country; }
36     @Override
37     public String toString() {
38         return name + " (" + country + ")";
39     }
40 }

```

The program prints

```

CH -> 2
UK -> 3
PL -> 3
US -> 2

```

The example below demonstrates combining predicates, and also **filter** function:

Listing 80

LDB-Predicates/Predicates.java

```

1 import java.util.List;
2 import java.util.function.Predicate;
3 import java.util.stream.Collectors;
4 import java.util.stream.Stream;
5
6 public class Predicates {
7     public static void main (String[] args) {
8         Predicate<Integer> p1 = e -> e%2 == 0;
9         Predicate<Integer> p = p1
10             .and(e -> e <= 10)
11             .or(e -> e == 19);
12         List<Integer> filteredList =
13             Stream.of(1,2,3,4,19,22,12)
14                 .filter(p)
15                 .collect(Collectors.toList());
16         filteredList.stream().forEach(System.out::println);

```

```

17      // prints 2, 4, 19
18  }
19 }

```

Several intermediate and terminal operations from previous examples are used below:

Listing 81

LDA-Streams/Streams.java

```

1  import java.util.Collections;
2  import java.util.Arrays;
3  import java.util.List;
4  import java.util.stream.Collectors;
5  import java.util.stream.IntStream;
6  import java.util.stream.Stream;
7
8  enum HairColor { BLACK, BROWN, BLOND, RED, WHITE };
9  enum EyeColor  { AMBER, BLUE, BROWN, GRAY, GREEN, HAZEL };
10 enum Sex        { WOMAN, MAN };
11
12 public class Streams {
13     public static void main(String... args) {
14         System.out.println("*Sorting by length of name");
15         Stream.of("Alice", "Margot", "Mary", "Sue")
16             .sorted((a,b) ->
17                 Integer.compare(a.length(),b.length()))
18             .forEach(System.out::println);
19
20         System.out.println("*Summing ints...");
21         int sum = IntStream.of(3,2,9,12,8,4)
22             .filter(n -> n%2 == 0)
23             .map(n -> 3*n+1)
24             .sorted()
25             .peek(n -> System.out.print(n+" "))
26             .sum();
27         System.out.println("\nSum = " + sum);
28
29         List<Person> listp = Arrays.asList(
30             new Person("Ann",Sex.WOMAN,
31                 HairColor.BLOND,EyeColor.BLUE),
32             new Person("Joe",Sex.MAN,
33                 HairColor.BLACK,EyeColor.BROWN),
34             new Person("Sue",Sex.WOMAN,
35                 HairColor.RED,EyeColor.HAZEL),
36             new Person("Ben",Sex.MAN,
37                 HairColor.BROWN,EyeColor.GREEN),
38             new Person("Bea",Sex.WOMAN,
39                 HairColor.WHITE,EyeColor.GRAY)
40         );
41

```

```

42     System.out.println("*Women's names...");
43     String womenNames =
44         listp.stream()
45             .filter(p -> p.getSex() == Sex.WOMAN)
46             .map(Person::getName)
47             .collect(Collectors.joining(", "));
48     System.out.println("Women: " + womenNames);
49
50     System.out.println("*Counting men...");
51     long menCount =
52         listp.stream()
53             .filter(p -> p.getSex() == Sex.MAN)
54             .count();
55     System.out.println("No. of men: " + menCount);
56
57     System.out.println("*Names staring with 'B'");
58     String nameB =
59         listp.stream()
60             .filter(p -> p.getName().charAt(0) == 'B')
61             .map(Object::toString)
62             .collect(Collectors.joining("\n"));
63     System.out.println(nameB);
64 }
65 }
66
67 class Person {
68     private final String    name;
69     private final Sex       sex;
70     private final HairColor hairColor;
71     private final EyeColor  eyeColor;
72     public Person(String n, Sex g,
73         HairColor hc, EyeColor ec) {
74         name      = n;
75         sex       = g;
76         hairColor = hc;
77         eyeColor  = ec;
78     }
79     public String getName() { return name; }
80     public Sex    getSex()  { return sex; }
81     public HairColor getHairColor() { return hairColor; }
82     public EyeColor getEyeColor() { return eyeColor; }
83
84     @Override
85     public String toString() {
86         return (sex == Sex.WOMAN ? "Mrs " : "Mr ") +
87             name + " (hair:" + hairColor + ", eyes:" +
88             eyeColor + ")";
89     }
90 }

```

The program prints

```
*Sorting by length of name
Sue
Mary
Alice
Margot
*Summing ints...
7 13 25 37
Sum = 82
*Women's names...
Women: Ann, Sue, Bea
*Counting men...
No. of men: 2
*Names starting with 'B'
Mr Ben (hair:BROWN, eyes:GREEN)
Mrs Bea (hair:WHITE, eyes:GRAY)
```

Next example demonstrates **map**, references to a constructor (also, to a ‘constructor’ of an array) and the **toArray** terminal operation.

Listing 82

LDH-RefConstr/RefConstr.java

```
1 import java.util.Arrays;
2 import java.util.stream.Stream;
3
4 public class RefConstr {
5     public static void main (String[] args) {
6         String[] names = {"Ada", "Bea", "Sue", "Lea" };
7         Person[] persons =
8             Stream.of(names)
9                 .map(Person::new)
10                .toArray(Person[]::new); // otherwise Object[]
11         System.out.println(Arrays.toString(persons));
12     }
13 }
14
15 class Person {
16     private String name;
17     Person(String n)      { name = n; }
18     public String getName() { return name; }
19     @Override
20     public String toString(){ return "Miss " + name; }
21 }
```

The program prints

```
[Miss Ada, Miss Bea, Miss Sue, Miss Lea]
```

The final example demonstrates **flatMap**, which ‘flattens’ a stream of streams into one stream, consisting of all elements from these individual streams:

Listing 83

LDI-FlatMap/Flat.java

```
1 import java.io.IOException;
2 import java.nio.file.Files;
3 import java.nio.file.Paths;
4 import java.util.Comparator;
5 import java.util.stream.Collectors;
6 import java.util.stream.Stream;
7 import static java.nio.charset.StandardCharsets.UTF_8;
8
9 public class Flat {
10     public static void main(String[] args) {
11         String result = null;
12         try (Stream<String> stream =
13             Files.lines(Paths.get("Flat.java"), UTF_8))
14         {
15             result =
16                 stream
17                     .flatMap(l -> Stream.of(l.split("\\P{L}+")))
18                     .filter(w -> w.length() > 9)
19                     .distinct()
20                     .sorted(Comparator.comparing(
21                         String::length).reversed())
22                     .collect(Collectors.joining(", "));
23         } catch (IOException ignore) { }
24         System.out.println(result);
25     }
26 }
```

The program prints

StandardCharsets, IOException, Comparator, Collectors

Method references

Quite often, when we override a method of a functional interface, its implementation reduces to invoking another method that already exists in a class. In such situations we can pass the reference to this method and the compiler will do the rest all by itself. For example, suppose we have a stream `stream` of objects of type `AClass`; we can terminate the pipeline of operations on the stream with `forEach` which then expects an object implementing `Consumer<AClass>`. Suppose, we just want to print elements of the stream — we can achieve this by writing

```
stream.forEach(e -> System.out.println(e))
```

or, in an abbreviated form, we just pass a method reference

```
stream.forEach(System.out::println)
```

In the latter case, we are telling the compiler *take elements of the stream and pass them, one by one, to the method indicated as an argument*. Notice, that in a method reference the name of the method must be preceded by a double colon and there are no parentheses after the name, because we don't invoke this function here; it is just a reference to the method itself. As `println` is a non-static method, it must be called on an object, so in front of the method reference we specify the object it is to be invoked on (in this case it is `System.out`).

There are three main forms of method references:

- `anObject::nonstaticMethod`
- `AClass::staticMethod`
- `AClass::nonstaticMethod`

In each case arguments for methods are needed, and also, in the third case, an object which will be the receiver of the invocation.

In the first case, `anObject::nonstaticMethod`, arguments will be passed as the argument to the method, so it is essentially equivalent to a lambda

```
e -> anObject.nonstaticMethod(e)
(e,f) -> anObject.nonstaticMethod(e,f)
```

depending of the number of arguments expected. Note that there can be several overloaded versions of the method: compiler will select the one matching the number and types of arguments passed. We have already seen an example: `System.out::println` corresponds to

```
e -> System.out.println(e)
```

In the second case, we can use a static method if the number and type of arguments matches the number and type of arguments of the static method. For example, if `Function<Double,Double,Double>` (or `BinaryOperator<Double>`) is expected in a given context, we can pass just `Math::pow`; this will be equivalent to passing the the lambda

```
(x, y) -> Math.pow(x,y)
```

In the third case, the first argument becomes the target (receiver) of the method and the remaining arguments are passed to the method as arguments. For example, **String::compareToIgnoreCase** corresponds to

```
(x, y) -> x.compareToIgnoreCase(y).
```

Suppose you want to sort an array of **Strings** (say, `strings`) ignoring the case. You can call **sort** which expects a **Comparator** that will be called with two arguments; you can pass the method reference

```
Arrays.sort(strings, String::compareToIgnoreCase)
```

Let us consider an example:

Listing 84

LCP-MethRefs/MethRefs.java

```

1 import java.util.Arrays;
2 import java.util.Collections;
3 import java.util.Iterator;
4 import java.util.List;
5
6 public class MethRefs {
7     List<String> list = Arrays.asList(
8         "Zoe", "kate", "Cindy", "barbra");
9     public static void main(String[] args) {
10         new MethRefs();
11     }
12
13     public MethRefs() {
14         // case anObject::nonstaticMethod
15         // Iterable<String> expected
16         Iterable<String> iterObj = this::getIter;
17         for (String s : iterObj) System.out.print(s + " ");
18         System.out.println();
19
20         // case AClass::staticMethod
21         // Runnable expected
22         Thread t = new Thread(MethRefs::tenFibos);
23         t.start();
24         try {
25             t.join();
26         } catch (InterruptedException ignore) { }
27
28         // case AClass::nonstaticMethod
29         // Comparator<String> expected
30         Collections.sort(list, String::compareTo);
31         System.out.println(list);
32         Collections.sort(list, String::compareToIgnoreCase);
33         System.out.println(list);
34     }
35
36     public static void tenFibos() {

```

```

37      // prints first 10 Fibonacci numbers
38      StringBuilder sb = new StringBuilder("0, 1");
39      int a = 0, b = 1;
40      for (int i = 0; i < 8; ++i) {
41          b += a;
42          a = b - a;
43          sb.append(", " + b);
44      }
45      System.out.println(sb);
46  }
47
48  public Iterator<String> getIter() {
49      return list.iterator();
50  }
51 }

```

which prints

```

Zoe kate Cindy barbra
0, 1, 1, 2, 3, 5, 8, 13, 21, 34
[Cindy, Zoe, barbra, kate]
[barbra, Cindy, kate, Zoe]

```

More examples can be found in Listing 75, Listing 77, Listing 78 and other listings from section 7 on p. 92.

Method references may also refer to constructors, even ‘constructors’ of arrays. The syntax is as follows:

- `AClass::new`
- `AType[]::new`

where in the second case **AType** may also be a primitive type (like `int`). In the first case, arguments are passed to a constructor, and its type decides which constructor will be used (if they are overloaded).

In the second case, there can be only one argument, a non-negative integer, which will be used as the size of the array created. There are also functions that accept a reference to an array constructor specifying type of the array to be created, which would otherwise be `Object[]`. For examples — see Listing 74 and Listing 82.

Section 9

Functional interfaces

Functional interfaces are those which declare *only one abstract* method, although they may contain definitions of default methods (marked with the keyword **default**) and static functions. When defining a functional interface, we should, although it is not strictly required, mark them with the annotation **FunctionalInterface** — the compiler will then check if the class definition actually defines a functional interface; e.g.,

```
@FunctionalInterface
interface Calc {
    double calculate(double d);
}
```

The standard library (in package **java.util.function**) defines several functional interfaces. The definitions are generic, i.e., they are expressed in terms of type parameters which may correspond to different (object) types. There are also versions with primitive types — **int**, **long**, **double** and **boolean**.

Let us briefly mention functional interfaces from the standard library.

9.1 Consumers

Consumers represent operations which accept (‘consume’) one or two arguments but do not return anything; therefore they are used for their side effects.

Interface **Consumer<T>** declares one abstract method of type **void** taking one argument:

```
void accept(T t)
```

where **T** denotes any *object* type. There are also versions for arguments of primitive types: **int**, **long** or **double**

- **IntConsumer** $\implies$ **void** accept(**int** t)
- **LongConsumer** $\implies$ **void** accept(**long** t)
- **DoubleConsumer** $\implies$ **void** accept(**double** t)

Interface **BiConsumer<T,U>** declares one abstract method of type **void** taking two arguments:

```
void accept(T t, U u)
```

where **T** and **U** denote any *object* types. There are also versions with one of the arguments, the *second*, of a primitive type: **int**, **long** or **double**

- **ObjIntConsumer<T>** $\implies$ **void** accept(T t, **int** u)
- **ObjLongConsumer<T>** $\implies$ **void** accept(T t, **long** u)
- **ObjDoubleConsumer<T>** $\implies$ **void** accept(T t, **double** u)

9.2 Functions

Functions represent operations which accept one or two arguments of some (possibly different) types and return a value of a certain type, which may be different from types of arguments (return types are conventionally denoted by the letter *R*).

Interface **Function**<*T*,*R*> declares one abstract method taking one argument and returning a value

R apply(*T* *t*)

where *T* and *R* denote any *object* types. There are also versions for arguments of primitive types: **int**, **long** or **double**

- **IntFunction**<*R*> $\implies$ *R* apply(**int** *t*)
- **LongFunction**<*R*> $\implies$ *R* apply(**long** *t*)
- **DoubleFunction**<*R*> $\implies$ *R* apply(**double** *t*)

Other versions take argument of an object type but return values of primitive types: **int**, **long** or **double** (note different names of their abstract methods!)

- **ToIntFunction**<*T*> $\implies$ **int** applyAsInt(*T* *t*)
- **ToLongFunction**<*T*> $\implies$ **long** applyAsLong(*T* *t*)
- **ToDoubleFunction**<*T*> $\implies$ **double** applyAsDouble(*T* *t*)

Finally, there are versions with argument and return value of (different) primitive types (note different names of their abstract methods!)

- **IntToLongFunction** $\implies$ **long** applyAsLong(**int** *t*)
- **IntToDoubleFunction** $\implies$ **double** applyAsDouble(**int** *t*)
- **LongToIntFunction** $\implies$ **int** applyAsInt(**long** *t*)
- **LongToDoubleFunction** $\implies$ **double** applyAsDouble(**long** *t*)
- **DoubleToIntFunction** $\implies$ **int** applyAsInt(**double** *t*)
- **DoubleToLongFunction** $\implies$ **long** applyAsLong(**double** *t*)

The cases when both types are the same will be handled by the interface **Operator**.

Interface **BiFunction**<*T*,*U*,*R*> declares one abstract method taking two arguments and returning a value

R apply(*T* *t*, *U* *u*)

where *T*, *U* and *R* denote any *object* types. There are also versions for return value of primitive type: **int**, **long** or **double** (note different names of their abstract methods!)

- **ToIntBiFunction**<*T*,*U*> $\implies$ **int** applyAsInt(*T* *t*, *U* *u*)
- **ToLongBiFunction**<*T*,*U*> $\implies$ **long** applyAsLong(*T* *t*, *U* *u*)
- **ToDoubleBiFunction**<*T*,*U*> $\implies$ **double** applyAsDouble(*T* *t*, *U* *u*)

9.3 Operators

Operators represent functions, for which the return type and the types of argument(s) are all the same (like for ‘normal’ operators: addition, multiplication, etc.). They fall into two categories: unary operators (with one argument) and binary operators (with two arguments). This interface extends **Function**, so the abstract methods have the same names as the corresponding functions.

Interface **UnaryOperator**<*T*> represents an operation on a single argument that produces a result of the same type as that of the argument; the abstract method is

```
T apply(T t)
```

where **T** is any *object* type. There are also versions for primitive types (note different names of their abstract methods!)

- **IntUnaryOperator** $\Rightarrow$ `int applyAsInt(int t)`
- **LongUnaryOperator** $\Rightarrow$ `long applyAsLong(long t)`
- **DoubleUnaryOperator** $\Rightarrow$ `double applyAsDouble(double t)`

Interface **BinaryOperator<T>** represents an operation with two arguments and returning a result: all of the same type (this interface extends **BiFunction**). The abstract method is therefore

```
T apply(T t1, T t2)
```

where **T** is any *object* type. There are versions for primitive types (note different names of their abstract methods!)

- **IntBinaryOperator** $\Rightarrow$ `int applyAsInt(int t1, int t2)`
- **LongBinaryOperator** $\Rightarrow$ `long applyAsLong(long t1, long t2)`
- **DoubleBinaryOperator** $\Rightarrow$ `double applyAsDouble(double t1, double t2)`

9.4 Predicates

Predicates are functions returning a logical value: either **true** or **false**.

Interface **Predicate<T>** represents a predicate with one argument

```
boolean test(T t)
```

where **T** is any *object* type. There are versions for primitive types:

- **IntPredicate** $\Rightarrow$ `boolean test(int t)`
- **LongPredicate** $\Rightarrow$ `boolean test(long t)`
- **DoublePredicate** $\Rightarrow$ `boolean test(double t)`

Interface **BiPredicate<T,U>** represents a predicate with two arguments and its abstract method is

```
boolean test(T t, U u)
```

9.5 Suppliers

Suppliers represent functions which do not take any argument but return a value.

Interface **Supplier<T>** declares an abstract method

```
T get()
```

where **T** is any *object* type. There are versions for primitive types (note different names of their abstract methods!)

- **BooleanSupplier** $\Rightarrow$ `boolean getAsBoolean()`
- **IntSupplier** $\Rightarrow$ `int getAsInt()`
- **LongSupplier** $\Rightarrow$ `long getAsLong()`
- **DoubleSupplier** $\Rightarrow$ `double getAsDouble()`

9.6 Example

The static function **mapFilter** in the program below expects a list, a **Predicate** and a **Function**. The predicate selects from a list elements satisfying the predicate, while the function transforms them to another type:

Listing 85

EMC-FInterfs/FInterfs.java

```
1 import java.util.ArrayList;
2 import java.util.Arrays;
3 import java.util.List;
4 import java.util.function.Function;
5 import java.util.function.Predicate;
6 public class FInterfs {
7     public static void main(String[] args) {
8         List<String> ls = Arrays.asList(
9             "Jane", "Sue", "Alice", "Kim", "Cecilia");
10        List<Character> lc = mapFilter(
11            ls, s -> s.length() > 3, s -> s.charAt(0));
12        System.out.println(lc);
13    }
14    static <T,R> List<R> mapFilter(
15        List<T> list, Predicate<T> p, Function<T,R> f) {
16        List<R> n = new ArrayList<>();
17        for (T e : list) if(p.test(e)) n.add(f.apply(e));
18        return n;
19    }
20 }
```

In the example above the predicate selects only strings longer than three characters, while the function transforms **String** into **Character**. The program prints [J, A, C].

Listings

List of listings

1	GXP-Split/Splitting.java	4
2	GXX-RegGroups/RegGroups.java	7
3	GXR-RegExNames/Name.java	11
4	GXR-RegExNames/RegEx.java	12
5	GXU-Regexes/Regexes.java	14
6	GXW-REExplorer/REExplorer.java	15
7	GXY-RegFind/RegFind.java	18
8	BGU-SingList/Node.java	21
9	BGU-SingList/MyList.java	21
10	BGU-SingList/Main.java	22
11	BGW-Stack/Node.java	23
12	BGW-Stack/MyStack.java	23
13	BGW-Stack/Main.java	24
14	BGZ-ArrStack/ArrStack.java	24
15	BGZ-ArrStack/Main.java	25
16	BGY-SimpQueue/MyQueue.java	26
17	BGY-SimpQueue/Main.java	26
18	EPZ-AbsFig/Figure.java	28
19	EPZ-AbsFig/Circle.java	29
20	EPZ-AbsFig/Rectangle.java	29
21	EPZ-AbsFig/Main.java	30
22	ELJ-InterStack/MyStacks.java	31
23	ELS-DefIntSimple/MyCompar.java	33
24	ELS-DefIntSimple/CompVal.java	33
25	ELS-DefIntSimple/CompDigits.java	34
26	ELS-DefIntSimple/CompValRev.java	34
27	ELS-DefIntSimple/Main.java	34
28	ELL-InterFun/InterFun.java	36
29	EQA-AbstractFigs/Figure.java	37
30	EQA-AbstractFigs/Circle.java	38
31	EQA-AbstractFigs/Rectangle.java	38
32	EQA-AbstractFigs/Main.java	39
33	ELP-Comps2/Person.java	40
34	ELP-Comps2/Comparators.java	41
35	ELP-Comps2/Main.java	41
36	ELM-Comps1/Person.java	43
37	ELM-Comps1/CompPerson.java	44
38	ELM-Comps1/Main.java	44
39	ELH-OutInn/OutInn.java	46
40	GMC-StackSimple/MyStack.java	48
41	GMC-StackSimple/StackSimple.java	48
42	ELI-Anon/Anon.java	49
43	EQE-AbstractAnimals/Animal.java	50
44	EQE-AbstractAnimals/Main.java	50
45	EQH-Closure/Closure.java	51
46	EMD-InterF/InterF.java	53
47	ELU-FInter/FInter.java	54

48	ELK-SimpleInter/SimpleInter.java	55
49	GMA-GenerPair/Pair.java	58
50	GME-QueueGener/MyQueue.java	60
51	GME-QueueGener/QueueGener.java	61
52	ELZ-IFac/IFaces.java	62
53	ELW-LambdaInter/MyBiOp.java	63
54	ELX-FuncInter/FuncInter.java	64
55	EMA-IFacLam/IFacesLam.java	65
56	EMB-Compos/Composit.java	67
57	GMH-MinMaxGener/MinMaxGener.java	68
58	GMI-MinMaxMeth/MinMaxMeth.java	69
59	CYG-Enum1/EnumEx1.java	70
60	CYE-SimpEnum/Suit.java	72
61	CYE-SimpEnum/Rank.java	72
62	CYE-SimpEnum/Card.java	73
63	CYE-SimpEnum/Main.java	73
64	CYH-EnumMet/EnumEx2.java	74
65	CYI-EnumAbs/EnumEx3.java	76
66	CYK-EnumInterf/CompEnum.java	77
67	JIX-RangIter/RangIter.java	82
68	JIZ-IterIterable/IterIterable.java	83
69	HUG-Lists/AList.java	84
70	HUK-SimpleMap/ASimpleMap.java	86
71	AAR-EquToString/EquToString.java	89
72	HUM-HashEquals/Person.java	90
73	HUM-HashEquals/AHash.java	91
74	LDC-Lazy/Lazy.java	97
75	LDF-StreamGrep/StreamGrep.java	98
76	LDD-StreamMisc/StreamMisc.java	99
77	LDG-MethRefStr/MethRefStr.java	101
78	LDE-Grouping/Grouping.java	102
79	LDJ-GroupCount/GroupCount.java	103
80	LDB-Predicates/Predicates.java	104
81	LDA-Streams/Streams.java	105
82	LDH-RefConstr/RefConstr.java	107
83	LDI-FlatMap/Flat.java	108
84	LCP-MethRefs/MethRefs.java	110
85	EMC-FInterfs/FInterfs.java	115

Index

- abstract class, [28](#)
- abstract method, [28](#)
- anonymous class, [46](#), [49](#)
- ArrayList (class), [80](#)

- backreference, [8](#)
- BiConsumer (interface), [112](#)
- BiFunction (interface), [113](#)
- BinaryOperator (interface), [114](#)
- BiPredicate (interface), [114](#)
- boxing, [79](#)

- capturing group, [6](#)
- CASE_INSENSITIVE, [10](#)
- class
 - abstract, [28](#)
 - anonymous, [46](#), [49](#)
 - generic, [58](#)
 - inner, [46](#)
 - Matcher, [5](#)
 - nested, [26](#)
 - outer, [46](#)
 - parametrized, [58](#)
 - Pattern, [5](#)
- ClassTypeException, [60](#)
- Collection (interface), [79](#)
- Comparable (interface), [36](#)
- Comparator (interface), [39](#)
- Consumer (interface), [112](#)

- default method, [31](#)
- diamond operator, [58](#)
- DOTALL, [10](#)

- effectively final, [51](#)
- entry, [81](#)
- enum, [70](#)
- equals (method), [88](#)

- FIFO, [25](#)
- first in, first out, [25](#)
- folding, [62](#)
- Function (interface), [113](#)
- functional interface, [31](#), [52](#), [112](#)

- generic class, [58](#)
- greedy quantifier, [2](#)
- group, [6](#)

- hashCode (method), [88](#)
- HashMap (class), [81](#)
- HashSet (class), [80](#)

- inner class, [46](#)
- interface, [31](#)
 - BiConsumer, [112](#)
 - BiFunction, [113](#)
 - BinaryOperator, [114](#)
 - BiPredicate, [114](#)
 - Comparable, [36](#)
 - Comparator, [39](#)
 - Consumer, [112](#)
 - Function, [113](#)
 - functional, [31](#), [52](#), [112](#)
 - Iterable, [82](#)
 - Map.Entry, [88](#)
 - Predicate, [114](#)
 - Supplier, [114](#)
 - UnaryOperator, [113](#)
- intermediate operation, [92](#)
- iterable, [79](#)
- Iterable (interface), [79](#), [82](#)

- key, [80](#)

- lambda, [52](#)
- last in, first out, [23](#)
- LIFO, [23](#)
- LinkedList (class), [80](#)
- list, [20](#)
- List (interface), [80](#)

- Map (interface), [79](#), [80](#)
- Map.Entry (interface), [88](#)
- Matcher (class), [5](#)
- method
 - abstract, [28](#)
 - default, [31](#)
 - equals, [88](#)
 - hashCode, [88](#)
 - overriding, [88](#)
 - private in interface, [31](#)
 - static in interface, [31](#)
- method reference, [109](#)
- MULTILINE, [9](#)

- natural order, [37](#)

- nested class, [26](#)
- operation
 - intermediate, [92](#)
 - pipeline of, [92](#)
 - short-circuiting, [92](#)
 - stateful, [92](#)
 - stateless, [92](#)
 - terminal, [92](#)
- operator
 - diamond, [58](#)
- option
 - CASE_INSENSITIVE, [10](#)
 - DOTALL, [10](#)
 - MULTILINE, [9](#)
 - UNICODE_CASE, [10](#)
 - UNI-
CODE_CHARACTER_CLASS,
[11](#)
- option flags, [9](#)
- optional operation, [80](#)
- outer class, [46](#)
- overriding a method, [88](#)
- parametrized class, [58](#)
- Pattern (class), [5](#)
- pipeline of operations, [92](#)
- possessive quantifier, [3](#)
- Predicate (interface), [114](#)
- quantifier, [2](#)
 - greedy, [2](#)
 - possessive, [3](#)
 - reluctant, [3](#)
- queue, [25](#)
- raw type, [59](#)
- reference to method, [109](#)
- regex, [1](#)
- regular expression, [1](#)
- reluctant quantifier, [3](#)
- Set (interface), [80](#)
- short-circuiting operation, [92](#)
- singly linked list, [20](#)
- stack, [23](#)
- stateful operation, [92](#)
- stateless operation, [92](#)
- stream, [92](#)
- super, [29](#)
- superclass, [28](#)
- Supplier (interface), [114](#)
- terminal operation, [92](#)
- TreeMap (class), [81](#)
- TreeSet (class), [80](#)
- type erasure, [60](#)
- type parameter, [58](#)
- UnaryOperator (interface), [113](#)
- unboxing, [79](#)
- UNICODE_CASE, [10](#)
- UNICODE_CHARACTER_CLASS,
[11](#)
- value, [80](#)